

Binary Trees and Tree Data Structures

A complete course, from the first definition to LeetCode Medium

Contents

How to Use This Material	5
0.1 Who this is for	5
0.2 What you will be able to do at the end	5
0.3 The parts	5
0.4 The tree used throughout	6
0.5 How each part is organised	7
0.6 Notes for teaching from this material	7
0.7 Conventions used	8
1 Tree Fundamentals	9
1.1 Why trees exist	9
1.2 What is a tree	9
1.3 The tree we will use everywhere	10
1.4 Terminology	10
1.5 Binary tree	11
1.6 Special shapes of binary trees	11
1.7 Binary Search Tree versus Binary Tree	12
1.8 Representing a tree in C++	13
1.9 Why recursion fits trees so naturally	14
1.10 Common mistakes	16
1.11 Check yourself	16
2 Depth First Traversals	17
2.1 What traversal means	17
2.2 The one picture that explains all three	17
2.3 Preorder traversal	18
2.4 Inorder traversal	20
2.5 Postorder traversal	21
2.6 Comparing the three	23
2.7 A tiny exercise you should actually do	23
2.8 Check yourself	24
3 BFS and Level Order Traversal	25
3.1 The problem DFS cannot solve comfortably	25
3.2 Queue intuition	25
3.3 Basic level order, everything in one list	26
3.4 Level order as a vector of vectors	26
3.5 Variations built on the same template	28
3.6 BFS or DFS, how to decide	30
3.7 Common mistakes	30
3.8 Check yourself	30

4	Iterative Traversals	32
4.1	Why bother	32
4.2	What the call stack actually stores	32
4.3	Iterative preorder	33
4.4	Iterative inorder	34
4.5	Iterative postorder	35
4.6	Summary table	37
4.7	Common mistakes	38
4.8	Check yourself	38
5	Basic Tree Properties	39
5.1	One template, many problems	39
5.2	Maximum depth, also called height	39
5.3	Minimum depth	40
5.4	Count nodes	41
5.5	Count leaf nodes	41
5.6	Sum of all nodes	41
5.7	Maximum value in the tree	42
5.8	Search for a value	42
5.9	Same tree	43
5.10	Symmetric tree	43
5.11	The pattern behind all of this	44
5.12	Common mistakes	44
5.13	Check yourself	45
6	Bottom-Up Thinking	46
6.1	The idea in one line	46
6.2	Where the naive approach goes wrong	46
6.3	The fix: return it instead of recomputing it	47
6.4	Designing the return value	47
6.5	Two ways to carry extra information	48
6.6	A worked example: the whole pattern on our tree	49
6.7	When bottom-up is not the answer	49
6.8	Complexity, and why it comes out to $O(N)$	50
6.9	Common mistakes	50
6.10	Check yourself	50
7	Diameter of a Binary Tree	52
7.1	What diameter means	52
7.2	Height is not diameter	52
7.3	The key observation	53
7.4	Brute force, and why it is slow	53
7.5	The $O(N)$ solution	54
7.6	Dry run	54
7.7	Complexity	55
7.8	Variations you should be able to handle	55
7.9	Common mistakes	56
7.10	Check yourself	56

8	Balanced Binary Tree	57
8.1	What balanced means	57
8.2	Naive solution	57
8.3	The $O(N)$ solution: the sentinel trick	58
8.4	Dry run of the sentinel version	59
8.5	The pattern, stated generally	60
8.6	Complexity summary	60
8.7	Common mistakes	61
8.8	Check yourself	61
9	Path Problems	62
9.1	Three kinds of “path”	62
9.2	Root to leaf: the basic shape	62
9.3	Collecting the actual paths: backtracking	64
9.4	Path from the root to a specific node	65
9.5	Any to any: maximum path sum	65
9.6	Downward path starting anywhere	67
9.7	Common mistakes	67
9.8	Check yourself	67
10	Core LeetCode Patterns	68
10.1	Invert a binary tree	68
10.2	Merge two binary trees	68
10.3	Subtree of another tree	69
10.4	Lowest Common Ancestor	69
10.5	Build a tree from traversal arrays	71
10.6	Serialize and deserialize	72
10.7	Just enough BST	73
10.8	Common mistakes	74
10.9	Check yourself	74
11	Tree Views	75
11.1	The idea: give every node a coordinate	75
11.2	Why BFS is the natural tool	76
11.3	Top view	76
11.4	Bottom view	77
11.5	Vertical order traversal	77
11.6	Right view and left view	78
11.7	Boundary traversal	78
11.8	Choosing the data structure	80
11.9	Common mistakes	80
11.10	Check yourself	81
12	Morris Traversal	82
12.1	The question it answers	82
12.2	The key observation	82
12.3	The algorithm	83
12.4	C++	83
12.5	Dry run	84

12.6 Complexity	85
12.7 Morris preorder	85
12.8 Risks and honest drawbacks	86
12.9 What you are expected to know	86
12.10 Common mistakes	86
12.11 Check yourself	87
13 How to Think About Tree Problems	88
13.1 The ten questions	88
13.2 The decision guide	89
13.3 Pattern to problem map	89
13.4 Applying the checklist to three unseen problems	90
13.5 When you are stuck	91
13.6 Habits that separate strong students	91
13.7 Check yourself	91
14 Practice Problem Sets	92
14.1 How to use this set	92
14.2 Level 1: Absolute Beginner	92
14.3 Level 2: Easy	93
14.4 Level 3: Easy plus	93
14.5 Level 4: Medium	94
14.6 The three view problems	95
14.7 Must Solve	95
14.8 A four week plan	96
15 Revision Sheet	97
15.1 Terminology in one box	97
15.2 Node definition	97
15.3 The master template	97
15.4 Recursive traversals	98
15.5 Iterative traversals	98
15.6 BFS template	99
15.7 Height template	99
15.8 Bottom-up template	100
15.9 Diameter template	100
15.10 Balanced template	100
15.11 Path templates	100
15.12 View templates	101
15.13 Morris template	101
15.14 Complexity cheat sheet	102
15.15 Which approach should I think of first	103
15.16 The mistakes that cost the most marks	103
15.17 The four questions to ask on any new tree problem	103

How to Use This Material

0.1 Who this is for

Someone who has never studied trees. You should be comfortable with C++ basics: functions, pointers at a beginner level, vector, queue, stack, and simple recursion such as factorial. Nothing else is assumed. Recursion is reintroduced from scratch in Part 1 where it is needed.

0.2 What you will be able to do at the end

Open any LeetCode Easy tree problem and solve it in ten minutes. Open a typical Medium tree problem and know within a minute which of five patterns it belongs to, then implement it. More importantly, look at a problem you have never seen and ask the right questions:

What information does each subtree need to return?

Is this DFS, BFS, or bottom-up?

What should my base case be?

What do I combine from the left and the right?

Memorising twenty solutions is not the goal, and does not work. Recognising five patterns does.

0.3 The parts

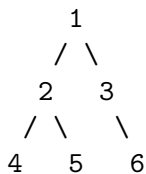
Part	Topic	What it gives you
1	Tree Fundamentals	vocabulary, TreeNode, why recursion fits trees
2	Recursive DFS	preorder, inorder, postorder, and when each is used
3	BFS and Level Order	the queue template and all its variations
4	Iterative Traversals	replacing the call stack with your own stack
5	Basic Tree Properties	your first ten recursive functions
6	Bottom-Up Thinking	returning information upward, avoiding $O(N^2)$
7	Diameter	the classic bottom-up problem
8	Balanced Binary Tree	the sentinel value pattern
9	Path Problems	root to leaf, backtracking, maximum path sum

Part	Topic	What it gives you
10	Core LeetCode Patterns	invert, subtree, LCA, construction, serialization, BST
11	Tree Views	horizontal distance, top, bottom, vertical, boundary
12	Morris Traversal	inorder in O(1) extra space
13	How to Think About Tree Problems	the pattern recognition checklist
14	Practice Problem Sets	graded problems with hints, no solutions
15	Revision Sheet	every template on a few pages

Read them in order. Parts 5, 6 and 7 are the spine of the whole course. If you are short of time, never skip those.

0.4 The tree used throughout

Every part uses the same example tree so that you build familiarity with one shape instead of re-reading a new diagram every page.



```

preorder    : 1 2 4 5 3 6
inorder     : 4 2 5 1 3 6
postorder   : 4 5 2 6 3 1
level order : 1 | 2 3 | 4 5 6
height      : 3 nodes, 2 edges
diameter    : 4 edges (4 -> 2 -> 1 -> 3 -> 6)
top view    : 4 2 1 3 6
bottom view : 4 2 5 3 6
right view  : 1 3 6
boundary    : 1 2 4 5 6 3

```

By the end of Part 11 you should be able to produce that whole block from the picture, from memory, in under two minutes. That is a good self-test.

Keep this in your practice file:

```

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
}

```

```
};

TreeNode* buildSampleTree() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->right = new TreeNode(6);
    return root;
}
```

Run every algorithm you learn on this tree locally. Reading code is not learning code.

0.5 How each part is organised

Every major topic follows the same shape:

1. what the thing means, in plain words
2. a picture
3. the algorithm in pseudocode
4. C++ implementation, LeetCode style
5. a dry run on a small tree
6. time and space complexity, with the reason
7. common mistakes
8. Check Yourself questions

The Check Yourself questions at the end of each part are not decoration. If you cannot answer them, reread that part rather than moving on. They are also a ready-made oral quiz for classroom use.

0.6 Notes for teaching from this material

- Parts 1 to 3 are one class each. Part 1 needs the full session, since terminology plus the recursion connection is a lot for a beginner.
- Parts 5 and 6 are best taught together, live-coding six functions in a row so the students see the template repeat.
- Part 7 works well as a discussion: ask the class for the brute force first, let them find the repeated work themselves, then show the rewrite. That moment is the whole point of the course.
- Part 12 (Morris) can be skipped for a weaker batch without damaging anything else.
- Part 13 is best taught after students have failed at a few Medium problems. It lands much harder then.
- Part 14 gives four weeks of homework. Part 15 is the handout for the day before an interview or contest.

0.7 Conventions used

- Code is C++, in the style LeetCode expects, with `NULL` rather than `nullptr` for readability at the whiteboard. Both are fine.
- Heights are counted in **nodes** in code (empty tree gives 0) and in **edges** in prose, and every place where this matters is called out explicitly.
- N is the number of nodes, H is the height, W is the maximum width of a level.
- Problem numbers in brackets refer to LeetCode. Three view problems come from GeeksforGeeks and are marked.

Part 1

Tree Fundamentals

Binary Trees and Tree Data Structures

1.1 Why trees exist

Arrays, vectors and linked lists all store data in a straight line. That works when the data itself is a line: marks of 50 students, characters of a string, prices over 30 days.

A lot of real data is not a line. It is a hierarchy.

```
Company
|-- Engineering
|   |-- Backend
|   +-- Frontend
+-- Sales
    |-- India
    +-- Europe
```

Folders inside folders, an HTML page inside body inside html, a tournament bracket, a family lineage, the decision steps of a game. In all of these, one item leads to several items below it, and each of those again leads to more. You cannot express that with a single line of memory without a lot of pain.

There is a second reason, a purely practical one. A sorted array searches in $O(\log N)$ but inserting in the middle costs $O(N)$. A linked list inserts in $O(1)$ but searches in $O(N)$. A balanced search tree gives you both search and insert in $O(\log N)$. Databases, file systems, `std::map` and `std::set` are all built on trees for exactly this reason.

So trees exist for two reasons:

1. Some data is naturally hierarchical.
2. Some operations are only fast when data is arranged in a branching structure.

For this course we care mostly about the first reason, because that is what LeetCode tree problems test.

1.2 What is a tree

A tree is a set of **nodes** connected by **edges**, with three rules:

1. One node is marked special. It is called the **root**, and it is drawn at the top.
2. Every other node has exactly one node directly above it (its parent).

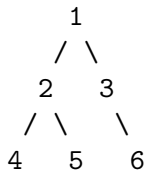
3. There are no cycles. You can never walk down and come back to where you started.

Because of rule 2 and 3, a tree with N nodes always has exactly $N - 1$ edges. Every node except the root consumes exactly one edge to attach itself to its parent.

Computer science trees are drawn upside down. Root on top, leaves at the bottom. Get used to it now, every diagram in this material follows that convention.

1.3 The tree we will use everywhere

Here is the tree that will appear again and again in this material. Memorise its shape. When a new algorithm is introduced you should be able to predict its output on this tree before reading the answer.



Node 1 is the root. Nodes 4, 5 and 6 have no children. Node 3 has only a right child.

1.4 Terminology

Everything below refers to the tree above.

Node: a single element. It holds a value and links to its children.

Edge: the link between a parent and a child. The pair (2, 4) is an edge.

Root: the topmost node, node 1. It has no parent.

Parent: node 2 is the parent of 4 and 5.

Child: 4 and 5 are children of 2. In a binary tree we say left child and right child, and the difference matters.

Siblings: nodes with the same parent. 4 and 5 are siblings. 2 and 3 are siblings.

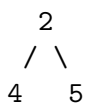
Leaf: a node with no children. Here: 4, 5, 6. A leaf is a node where both `left` and `right` are `NULL`.

Internal node: any node that is not a leaf. Here: 1, 2, 3.

Ancestor: any node on the path from a node up to the root. Ancestors of 5 are 2 and 1.

Descendant: everything below a node. Descendants of 2 are 4 and 5.

Subtree: a node together with all of its descendants. The subtree rooted at 2 is



This word is the most important one on the page. A subtree is itself a complete, valid tree. That single fact is what makes recursion work on trees.

Level: nodes at the same distance from the root form a level. Level 0 = {1}, level 1 = {2, 3}, level 2 = {4, 5, 6}.

Depth of a node: number of edges from the root down to that node. $\text{Depth}(1) = 0$, $\text{depth}(2) = 1$, $\text{depth}(4) = 2$. Depth is measured looking upward, toward the root.

Height of a node: number of edges on the longest path from that node down to a leaf. $\text{Height}(4) = 0$, $\text{height}(2) = 1$, $\text{height}(1) = 2$. Height is measured looking downward.

Height of the tree: the height of its root. Here, 2.

1.4.1 Depth versus height, and the counting trap

Depth and height are measured in opposite directions. A leaf has height 0. The root has depth 0.

There is a second trap. Some books count **edges**, some count **nodes**. A single node alone has height 0 by the edge definition and height 1 by the node definition. Both are used in the real world.

LeetCode uses the node count. Problem 104, “Maximum Depth of Binary Tree”, says the depth of the tree above is 3, not 2.

Rule for this course:

- When we write a LeetCode style function called `maxDepth` or `height`, we count **nodes**, so an empty tree returns 0 and a single node returns 1.
- When we discuss theory such as “the height of a perfect tree is $\log N$ ”, we count **edges**.

Whenever you sit down to code, decide which one you are using in the first ten seconds, and make the base case agree with it. Almost every off-by-one bug in tree code comes from skipping this step.

1.5 Binary tree

A **binary tree** is a tree where every node has at most two children, and the two children are distinguishable: one is the left child, one is the right child.

“Distinguishable” is not a detail. These two trees are different trees:



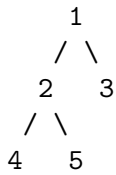
The first has 2 as a left child, the second as a right child. Their inorder traversals differ, their views differ, and a function like “is this tree symmetric” will give different answers.

Almost every interview and contest tree problem is about binary trees, which is why we spend the whole course on them.

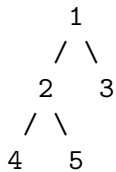
1.6 Special shapes of binary trees

You need these names to read problem statements. You do not need formal proofs.

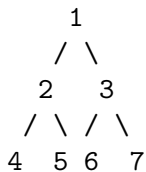
Full binary tree: every node has either 0 or 2 children. No node has exactly one child.



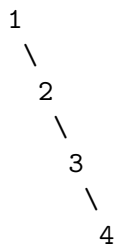
Complete binary tree: every level is completely filled except possibly the last, and the last level is filled from left to right with no gaps. This is the shape a heap uses, and it is the shape that lets you store a tree in a plain array.



Perfect binary tree: every internal node has 2 children and all leaves are on the same level.



Skewed or degenerate tree: every node has only one child. It is a linked list wearing a tree costume.



1.6.1 Why the shape matters

Two numbers you should be able to state instantly:

- A perfect tree of height h has $2^{(h+1)} - 1$ nodes. Level d holds at most 2^d nodes.
- Therefore a balanced tree with N nodes has height about $\log_2(N)$. For $N = 10^6$ that is about 20.

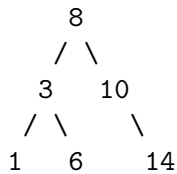
A skewed tree with N nodes has height $N - 1$.

Every recursive tree algorithm uses $O(H)$ stack space, where H is the height. On a balanced tree that is 20 stack frames. On a skewed tree of 10^5 nodes that is 10^5 stack frames, and on some judges that is a stack overflow. When you see “the tree can be skewed” in constraints, that is the hint.

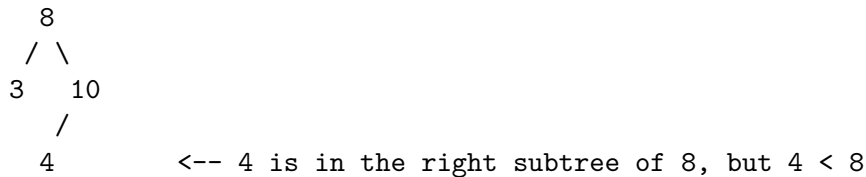
1.7 Binary Search Tree versus Binary Tree

A **Binary Search Tree (BST)** is a binary tree with one extra promise:

For every node, every value in its left subtree is smaller than the node, and every value in its right subtree is larger.



Note the word **every**. It is not enough that the immediate left child is smaller. The whole left subtree must be smaller. This tree is **not** a BST even though each node looks locally fine:



The single most useful consequence: **the inorder traversal of a BST is sorted**. We will prove that visually in Part 2.

For the rest of the course, assume a tree is a plain binary tree with arbitrary values unless the problem explicitly says BST. Students lose marks and interview points by assuming order that was never given.

1.8 Representing a tree in C++

This is the structure used by LeetCode and by everything in this material:

```

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
  
```

Three fields. A value, a pointer to the left child, a pointer to the right child. NULL means “there is no child there”.

Two observations that carry the whole course:

1. **A tree is just a pointer to its root.** There is no separate Tree object. When a function takes `TreeNode* root`, it has received an entire tree.
2. **Every node is the root of its own subtree.** So `root->left` is not “a child”, it is “a smaller tree”. Passing `root->left` into the same function is completely legal and completely meaningful.

That is the whole idea behind tree recursion. Nothing more.

1.8.1 Building our tree by hand

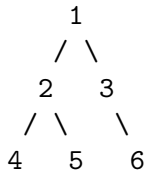
```

TreeNode* buildSampleTree() {
    TreeNode* root = new TreeNode(1);
  
```

```

root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);
root->right->right = new TreeNode(6);
return root;
}

```



Keep this function in your practice file. When you want to test any algorithm from this material locally, build this tree and run on it.

1.8.2 The array representation

A complete binary tree can be stored in a plain array with no pointers at all:

- node at index i has children at $2*i + 1$ and $2*i + 2$
- its parent is at $(i - 1) / 2$

```

index:  0  1  2  3  4  5  6
value:  1  2  3  4  5  6  7

```

This is how heaps and segment trees are built. For general binary trees it wastes memory (a skewed tree of 20 nodes would need an array of size 2^{20}), so LeetCode problems use pointers.

1.8.3 How LeetCode writes trees in the input

You will see input like:

```
[1,2,3,4,5,null,6]
```

That is level order, left to right, with `null` marking a missing child. Read it as:

```

level 0:  1
level 1:  2 3
level 2:  4 5 null 6

```

which is exactly our tree. You never have to parse this yourself in the editor, but you must be able to draw the tree from it on paper. Practice that until it is instant.

1.9 Why recursion fits trees so naturally

1.9.1 A quick recursion refresher

A recursive function calls itself on a smaller version of the same problem. It needs two parts:

1. a **base case**, small enough to answer directly
2. a **recursive step**, which solves a smaller input and builds the answer from it

```
int fact(int n) {
    if (n == 0) return 1;           // base case
    return n * fact(n - 1);        // smaller version of the same problem
}
```

When you call `fact(3)`, the machine does not magically know the answer. It stacks up pending work:

```
fact(3) waits for fact(2)
    fact(2) waits for fact(1)
        fact(1) waits for fact(0)
            fact(0) = 1                <- base case, now unwind
        fact(1) = 1 * 1 = 1
    fact(2) = 2 * 1 = 2
fact(3) = 3 * 2 = 6
```

Two things to notice, because they reappear in every tree problem:

- The going-down phase and the coming-back-up phase are different moments. You can do work in either one.
- Every pending call sits on the **call stack**, which costs memory.

1.9.2 Now the tree version

For an array you had to invent the smaller problem: $n - 1$, or “the rest of the array”. In a tree the smaller problem is already sitting there. `root->left` is a smaller tree. `root->right` is a smaller tree. The structure hands you the recursion for free.

So every tree function looks like this:

```
ReturnType solve(TreeNode* root) {
    if (root == NULL) return answerForEmptyTree;

    ReturnType L = solve(root->left);    // answer for the left subtree
    ReturnType R = solve(root->right);   // answer for the right subtree

    return combine(L, R, root->val);     // build my answer from theirs
}
```

Read that template again. Roughly seventy percent of binary tree problems on LeetCode are this template with a different `combine` and a different `ReturnType`. Part 5 and Part 6 are entirely about filling in those two blanks.

The base case is almost always `root == NULL`. Ask yourself: what is the correct answer for an empty tree? Number of nodes: 0. Height: 0. Sum: 0. Maximum value: negative infinity. Is it balanced: yes. That question, asked honestly, gives you the base case every single time.

1.9.3 What the calls look like on our tree

```
solve(1)
|-- solve(2)
|   |-- solve(4)
|   |   |-- solve(NULL)
|   |   +-- solve(NULL)
```

```

|    +-- solve(5)
|        |-- solve(NULL)
|        +-- solve(NULL)
+-- solve(3)
    |-- solve(NULL)
    +-- solve(6)
        |-- solve(NULL)
        +-- solve(NULL)

```

Every node is entered exactly once, so the work is $O(N)$ as long as `combine` is $O(1)$. The deepest chain of pending calls is the height of the tree, so the extra space is $O(H)$: $O(\log N)$ for a balanced tree, $O(N)$ in the worst case.

Learn to draw this diagram. When a recursive tree solution misbehaves, drawing the call tree for a 5 node example finds the bug faster than any debugger.

1.10 Common mistakes

- **Forgetting the NULL base case.** Then `root->val` dereferences a null pointer and you get a runtime error or a segfault. Write the `if (root == NULL)` line first, before anything else.
- **Checking the child instead of the node.** Writing `if (root->left == NULL) return ...` at the top means an empty tree still crashes, and the logic gets duplicated for both sides. Guard the node itself.
- **Mixing up height and depth.** Depth counts downward from the root, height counts upward from the leaves.
- **Mixing up edge counting and node counting.** Decide first, then write the base case to match.
- **Assuming a plain binary tree is a BST.** Only assume ordering when the problem says so.
- **Assuming a leaf means “one child is NULL”.** A leaf means **both** children are NULL. This one specific mistake breaks the Minimum Depth problem for almost everyone the first time.
- **Forgetting that the tree can be a single long chain.** Recursion depth equals height, not $\log N$.

1.11 Check yourself

1. Our sample tree has 6 nodes. How many edges does it have, and why can you answer without looking at the picture?
2. What is the height of node 3, and what is the depth of node 3?
3. A binary tree has height 4 counting edges. What is the largest number of nodes it can contain? What is the smallest?
4. Why is a subtree also a tree, and why does that matter for writing code?
5. For a function that counts the number of nodes, what should the answer be when `root == NULL`, and how did you decide?
6. Draw the tree for the LeetCode input `[3,9,20,null,null,15,7]`.

Part 2

Depth First Traversals

Preorder, Inorder, Postorder

2.1 What traversal means

Traversal means visiting every node exactly once in some fixed order. That is all.

An array has one obvious order: left to right. A tree does not. From node 1 you can go to 2 or to 3, and there is no single natural answer. Different orders are useful for different jobs, so we standardise a few of them and give them names.

Depth First Search (DFS) means: go as deep as possible down one branch before coming back and trying the other branch. It is what a person does in a maze when they follow one corridor to its end before backtracking.

Breadth First Search (BFS) means: finish an entire level before moving to the next. That is Part 3.

DFS on a binary tree comes in three flavours. They differ in exactly one thing: **when you record the current node relative to the two recursive calls.**

```
// preorder
visit(node); go(left); go(right);

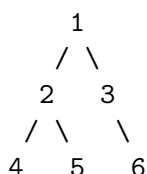
// inorder
go(left); visit(node); go(right);

// postorder
go(left); go(right); visit(node);
```

The names tell you where “root” sits: **pre** = root first, **in** = root in the middle, **post** = root last. In all three, left is always explored before right.

2.2 The one picture that explains all three

Imagine walking around the outside of the tree, keeping your left hand on the wall, starting and ending at the root.



During that walk you touch every node exactly **three** times:

1. when you arrive at it from above (going down into it)
2. when you come back up from its left subtree
3. when you come back up from its right subtree, just before leaving it for good

Now the three traversals are simply three choices of which touch counts:

Traversal	Record the node on which touch
Preorder	1st touch, on the way down
Inorder	2nd touch, between left and right
Postorder	3rd touch, on the way out

The results for our tree:

```
Preorder   : 1 2 4 5 3 6
Inorder    : 4 2 5 1 3 6
Postorder  : 4 5 2 6 3 1
```

Check those three lines against the picture until they feel obvious. Once this single picture is solid, iterative traversal and Morris traversal later become much less mysterious, because they are just different ways of remembering “which touch am I on”.

2.3 Preorder traversal

2.3.1 Meaning

Process the node, then everything on its left, then everything on its right. Root comes before its descendants, always.

2.3.2 Recursive algorithm

```
preorder(node):
    if node is empty: return
    record node
    preorder(node.left)
    preorder(node.right)
```

2.3.3 C++

```
void preorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    out.push_back(root->val);
    preorder(root->left, out);
    preorder(root->right, out);
}
```

The LeetCode wrapper (problem 144):

```
vector<int> preorderTraversal(TreeNode* root) {
    vector<int> out;
    preorder(root, out);
    return out;
}
```

Note the `vector<int>&`. The ampersand means all calls share one vector. Drop it and every call gets its own copy, the pushes are thrown away, and you return an empty answer. This is one of the most common beginner bugs in tree code.

2.3.4 Dry run

call preorder(1)	record 1	out = [1]
call preorder(2)	record 2	out = [1,2]
call preorder(4)	record 4	out = [1,2,4]
preorder(NULL)	return	
preorder(NULL)	return	
return from 4		
call preorder(5)	record 5	out = [1,2,4,5]
two NULL calls, return		
return from 5		
return from 2		
call preorder(3)	record 3	out = [1,2,4,5,3]
preorder(NULL)	return	
call preorder(6)	record 6	out = [1,2,4,5,3,6]
return from 6		
return from 3		
done		out = [1,2,4,5,3,6]

As a call tree:

```
pre(1)
|-- pre(2)
|   |-- pre(4)
|   +-- pre(5)
+-- pre(3)
    +-- pre(6)
```

Read the node values in the order the calls **start**, and you get the preorder.

2.3.5 Complexity

Time $O(N)$: every node is entered once and does $O(1)$ work. Space $O(H)$ for the call stack, where H is the height. $O(\log N)$ if balanced, $O(N)$ if skewed. The output vector is a separate $O(N)$, which we usually do not count as extra space since it is the answer.

2.3.6 When preorder is the right choice

- **Copying or cloning a tree.** You must create the parent before you can attach children to it.
- **Serialising a tree** so it can be rebuilt later. Preorder plus null markers is enough to reconstruct a tree uniquely.
- **Building a tree from preorder plus inorder arrays** (LeetCode 105). The first element of preorder

is always the root.

- **Passing information downward**, for example “here is the path from the root to you” or “here is the sum so far”. Anything the parent knows and the child needs is naturally handed over on the way down. This is the shape of all root-to-leaf path problems in Part 9.

The rule of thumb: preorder is the traversal for **top-down** information flow.

2.3.7 Common mistakes

- Forgetting & on the output vector.
- Putting the push_back after the two calls “because it looks tidier”. That is postorder, and the output changes.
- Checking if (root->left != NULL) preorder(root->left) instead of guarding inside the function. It works, but it doubles the number of conditions you must get right, and it breaks immediately for an empty tree.

2.4 Inorder traversal

2.4.1 Meaning

Everything on the left, then the node, then everything on the right.

2.4.2 Recursive algorithm

```
inorder(node):
    if node is empty: return
    inorder(node.left)
    record node
    inorder(node.right)
```

2.4.3 C++

```
void inorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    inorder(root->left, out);
    out.push_back(root->val);
    inorder(root->right, out);
}
```

2.4.4 Dry run

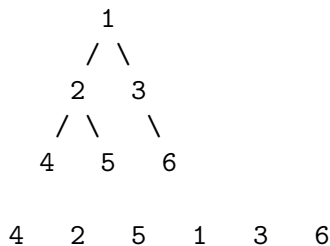
```
in(1)
  in(2)
    in(4)
      in(NULL) return
      record 4           out = [4]
      in(NULL) return
    record 2             out = [4,2]
  in(5)
    record 5             out = [4,2,5]
  record 1               out = [4,2,5,1]
```

```

in(3)
  in(NULL) return
  record 3          out = [4,2,5,1,3]
  in(6)
    record 6        out = [4,2,5,1,3,6]
done                out = [4,2,5,1,3,6]

```

A useful visual shortcut: drop every node straight down onto a horizontal line, keeping left nodes to the left. Reading that line from left to right gives the inorder sequence.



That is exactly the projection of the tree onto the x axis, and it is also the seed of the vertical order idea in Part 11.

2.4.5 Complexity

Time $O(N)$, space $O(H)$. Same as preorder.

2.4.6 When inorder is the right choice

- **Anything involving a BST.** The inorder traversal of a BST is sorted, ascending. This single fact solves a whole family of problems: validate BST (LeetCode 98), kth smallest element (230), minimum absolute difference (530), convert BST to sorted list, and more.
- **Finding the predecessor or successor of a node** in a BST.
- Recovering a tree when you are given the inorder array plus preorder or postorder.

Why is a BST's inorder sorted? By definition, everything in the left subtree is smaller than the node and everything in the right subtree is larger. Inorder prints exactly in that order: smaller stuff, node, larger stuff. Apply the same reasoning inside each subtree and the whole sequence is sorted.

2.4.7 Common mistakes

- Assuming inorder is sorted for a **general** binary tree. It is not. On our tree it is 4 2 5 1 3 6, which is nowhere near sorted. That property is a BST property, not a tree property.
- In “validate BST”, checking only `left->val < root->val < right->val`. That is a local check and it is wrong. Either do an inorder walk and confirm it is strictly increasing, or pass a valid (min, max) range downward.
- Losing the previous value while validating. You need to remember the previously printed node to compare against, usually a `TreeNode* prev` kept outside the recursion.

2.5 Postorder traversal

2.5.1 Meaning

Everything on the left, everything on the right, and only then the node itself. The root is always last.

2.5.2 Recursive algorithm

```
postorder(node):
    if node is empty: return
    postorder(node.left)
    postorder(node.right)
    record node
```

2.5.3 C++

```
void postorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    postorder(root->left, out);
    postorder(root->right, out);
    out.push_back(root->val);
}
```

2.5.4 Dry run

```
post(1)
  post(2)
    post(4) -> record 4      out = [4]
    post(5) -> record 5      out = [4,5]
    record 2                  out = [4,5,2]
  post(3)
    left is NULL
    post(6) -> record 6      out = [4,5,2,6]
    record 3                  out = [4,5,2,6,3]
  record 1                    out = [4,5,2,6,3,1]
```

Notice the pattern: a node is recorded only after **both** of its subtrees are completely finished. The root is recorded last, always.

2.5.5 Complexity

Time $O(N)$, space $O(H)$.

2.5.6 When postorder is the right choice

This is the most important traversal for problem solving, and students usually underestimate it.

Postorder is the traversal for **bottom-up** information flow. Use it whenever the answer at a node depends on answers computed inside its subtrees:

- height of a tree
- number of nodes, number of leaves, sum of nodes
- diameter
- is the tree balanced
- maximum path sum
- deleting a tree (you must free the children before the parent, otherwise you lose the pointers)

- evaluating an expression tree: compute both operands, then apply the operator

Every one of those has the same skeleton:

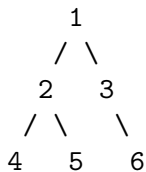
```
int solve(TreeNode* root) {
    if (root == NULL) return baseAnswer;
    int L = solve(root->left);
    int R = solve(root->right);
    return combine(L, R, root->val);    // this line is postorder position
}
```

If you understand that combine sits exactly where postorder records the node, you have understood Part 6 before we get there.

2.5.7 Common mistakes

- Recording the node before the recursive calls. That is preorder.
- Trying to build a postorder answer while descending. It cannot be done: at the moment you arrive at a node, you do not yet know anything about its subtrees.
- Writing the iterative version from memory without understanding it. That is Part 4, and it is the one traversal that genuinely needs care.

2.6 Comparing the three



Preorder	1 2 4 5 3 6	root first	top-down information
Inorder	4 2 5 1 3 6	root middle	BST and sorted order
Postorder	4 5 2 6 3 1	root last	bottom-up information

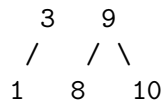
Facts worth remembering:

- In preorder, the **first** element is always the root.
- In postorder, the **last** element is always the root.
- Inorder alone cannot rebuild a tree, but inorder plus either of the other two can.
- All three visit every node once: $O(N)$ time, $O(H)$ space.
- Reversing a “root, right, left” preorder gives postorder. That trick is the basis of the easiest iterative postorder in Part 4.

2.7 A tiny exercise you should actually do

Take this tree:





Write down all three traversals before reading on.

Preorder : 7 3 1 9 8 10

Inorder : 1 3 7 8 9 10

Postorder : 1 3 8 10 9 7

The inorder came out sorted. That is not luck. This tree is a BST.

2.8 Check yourself

1. Which traversal gives you the root first, and why does that matter when copying a tree?
2. You are given the postorder array of a tree. Which element is the root, and what do you look for next?
3. Why is the space complexity $O(H)$ and not $O(N)$ for a balanced tree, even though there are N nodes?
4. A tree has inorder 4 2 5 1 3 6 and preorder 1 2 4 5 3 6. Which nodes are in the left subtree of the root, and how did you know?
5. You need to compute the height of every node. Which of the three traversals is the natural one, and why can it not be done in preorder?
6. What single line do you move to convert a preorder function into a postorder function?

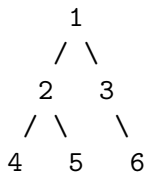
Part 3

BFS and Level Order Traversal

Processing a tree one level at a time

3.1 The problem DFS cannot solve comfortably

Look at our tree again.



Suppose the question is: “print the nodes level by level” or “what is the rightmost node on each level” or “which level has the largest sum”. DFS jumps from 4 straight to 5 and then all the way over to 3. Nodes on the same level are visited far apart in time. You can still do it with DFS by tracking depth, and sometimes we do, but the natural tool here is BFS.

BFS finishes an entire level before moving down. On our tree it produces:

```
level 0:  1
level 1:  2 3
level 2:  4 5 6
```

3.2 Queue intuition

BFS needs a **queue**: first in, first out. Think of a line at a ticket counter. The person who joined first is served first, and new arrivals go to the back.

```
queue<TreeNode*> q;
q.push(node);      // join the back of the line
q.front();          // who is at the front
q.pop();            // remove the front person
q.empty();          // is the line empty
q.size();           // how many are waiting
```

The algorithm in one sentence: **take the node at the front, note its value, and put its children at the back.**

Why does that produce level order? Because a node’s children always join the queue after every node of the current level has already joined. The queue never lets a deeper node overtake a shallower one.

If you want a mental image, the queue at any moment holds the “frontier”: the last part of one level plus the first part of the next.

3.3 Basic level order, everything in one list

```
vector<int> levelOrder(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* node = q.front();
        q.pop();

        out.push_back(node->val);

        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    return out;
}
```

3.3.1 Dry run

start	q = [1]		out = []
pop 1	push 2, 3	q = [2,3]	out = [1]
pop 2	push 4, 5	q = [3,4,5]	out = [1,2]
pop 3	push 6	q = [4,5,6]	out = [1,2,3]
pop 4	no children	q = [5,6]	out = [1,2,3,4]
pop 5	no children	q = [6]	out = [1,2,3,4,5]
pop 6	no children	q = []	out = [1,2,3,4,5,6]
queue empty, stop			

Every node enters the queue exactly once and leaves exactly once.

Time $O(N)$. Space $O(W)$, where W is the maximum width of the tree. In the worst case the last level of a perfect tree holds about $N/2$ nodes, so this is $O(N)$. Note the difference from DFS, whose space is $O(H)$. For a wide shallow tree DFS is cheaper, for a deep skewed tree BFS is cheaper.

3.4 Level order as a vector of vectors

This is LeetCode 102, and it is the version you will actually reuse. The output for our tree should be `[[1], [2,3], [4,5,6]]`.

The only new idea is one line: **before the inner loop starts, record how many nodes are currently in the queue. That count is exactly the size of the current level.**

```

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (root == NULL) return res;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int sz = q.size();           // size of the current level, snapshot it
        vector<int> level;

        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front();
            q.pop();

            level.push_back(node->val);

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

3.4.1 Why `int sz = q.size()` must come before the loop

During the inner loop you are pushing children, so `q.size()` keeps changing. If you write `for (int i = 0; i < q.size(); i++)` the condition is re-evaluated every iteration and you will happily consume the next level too, producing one giant level. Take the snapshot once. This is the single most common BFS bug.

3.4.2 Dry run

```

outer pass 1:  sz = 1,  q = [1]
               pop 1, level = [1], push 2,3
               res = [[1]]                                q = [2,3]

outer pass 2:  sz = 2,  q = [2,3]
               pop 2, level = [2], push 4,5
               pop 3, level = [2,3], push 6
               res = [[1],[2,3]]                            q = [4,5,6]

outer pass 3:  sz = 3,  q = [4,5,6]
               pop 4, pop 5, pop 6, no children
               res = [[1],[2,3],[4,5,6]]                    q = []

```

This template is the skeleton for every level based problem. Learn it once and only change what happens inside the inner loop.

3.5 Variations built on the same template

3.5.1 Level sums

```
vector<int> levelSums(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int sz = q.size();
        long long sum = 0;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            sum += node->val;
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back((int)sum);
    }
    return res;
}
```

For our tree: [1, 5, 15]. Use long long when values can be large and a level can be wide. LeetCode 1161 (maximum level sum) is this function plus an argmax.

3.5.2 Level maximum

Replace the accumulator:

```
int mx = INT_MIN;
// inside the inner loop
mx = max(mx, node->val);
```

For our tree: [1, 3, 6]. That is LeetCode 515.

3.5.3 Level averages

Same loop, keep sum and divide by sz. Use double, and accumulate in long long or double to avoid overflow. LeetCode 637.

3.5.4 Right side view

The right side view is what you see standing to the right of the tree: the **last** node of every level.

```
vector<int> rightSideView(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
```

```

    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        TreeNode* node = q.front(); q.pop();
        if (i == sz - 1) res.push_back(node->val);    // last of this level
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}
return res;
}

```

For our tree: 1 3 6. The left view is the same code with `i == 0`. For our tree: 1 2 4.

Note carefully: the right view is **not** “keep going right from the root”. On our tree that path is 1, 3, and then 3 has no right child, so you would miss 6 entirely. The right view is the last node of each level, wherever it comes from.

3.5.5 Zigzag level order

Left to right on even levels, right to left on odd levels (LeetCode 103). Do not use a stack and do not reverse the queue. Build the level normally, then reverse the level vector when needed:

```

bool leftToRight = true;
// after building the level vector:
if (!leftToRight) reverse(level.begin(), level.end());
res.push_back(level);
leftToRight = !leftToRight;

```

3.5.6 Bottom-up level order

LeetCode 107 wants levels from the bottom up. Build normally, then `reverse(res.begin(), res.end())`. Do not try to be clever.

3.5.7 Minimum depth

The depth of the nearest leaf. BFS is genuinely better here: the first leaf you meet is the answer, and you can stop immediately.

```

int minDepth(TreeNode* root) {
    if (root == NULL) return 0;
    queue<TreeNode*> q;
    q.push(root);
    int depth = 1;

    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            if (node->left == NULL && node->right == NULL) return depth;
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        depth++;
    }
}

```

```

    return depth;
}

```

On a tree with a million nodes where one leaf sits at depth 2, DFS explores everything and BFS stops almost immediately. When a problem says “nearest”, “minimum number of steps”, or “first level where something happens”, think BFS.

3.6 BFS or DFS, how to decide

Situation	Use
Answer depends on levels or distance from the root	BFS
Nearest or minimum depth style question	BFS
Right view, left view, level sums, zigzag	BFS
Answer is built from subtree answers	DFS, postorder
Path from root to some node	DFS, preorder
Height, diameter, balanced, path sums	DFS
Tree is very deep and narrow, recursion may overflow	BFS, or iterative DFS
Tree is very wide and shallow, memory matters	DFS

Almost anything BFS can do, DFS can also do by carrying the depth as a parameter. The right view, for instance, can be done with a “root, right, left” DFS that records the first node it meets at each new depth. Use whichever you can write correctly under pressure. For genuinely level based problems, BFS is usually shorter and easier to explain.

3.7 Common mistakes

- **Recomputing `q.size()` inside the loop condition.** Snapshot it into `sz` first.
- **Pushing NULL children.** Then `node->val` crashes later. Either guard on push (preferred) or guard on pop, but do not do neither.
- **Forgetting the empty tree.** `if (root == NULL) return {};` on the first line.
- **Using vector with `erase(begin())` as a queue.** That is $O(N)$ per removal and turns an $O(N)$ algorithm into $O(N^2)$. Use `queue<TreeNode*>`.
- **Calling `q.front()` after `q.pop()`.** Read the value first, then pop.
- **Trying to detect the end of a level by comparing values.** Values repeat. Use the size snapshot.
- **Thinking the right view means always going right.** It does not.

3.8 Check yourself

1. Why does taking a snapshot of the queue size cleanly separate the levels?
2. Our tree gives `[[1], [2,3], [4,5,6]]`. What would BFS give for a completely skewed tree of 4 nodes?

3. What is the worst case space of BFS, and which tree shape causes it?
4. Which is better for minimum depth, BFS or DFS, and why?
5. You need the leftmost node of the deepest level. Sketch a BFS that finds it with almost no extra code.
6. Can you produce a right side view with DFS? What visiting order would you use?

Part 4

Iterative Traversals

Replacing the call stack with your own stack

4.1 Why bother

Recursion already works and is shorter. So why learn the iterative versions?

1. **Interviewers ask for it.** “Now do it without recursion” is a standard follow-up.
2. **Deep trees overflow the call stack.** A skewed tree with 10^5 nodes means 10^5 pending calls. Your own stack lives on the heap and does not have that limit.
3. **You get control.** Pausing a traversal, resuming it later, or implementing a BST iterator (LeetCode 173) is easy with an explicit stack and impossible with plain recursion.
4. **It teaches you what recursion actually does.** After this part, Morris traversal in Part 12 will look like a natural next step instead of black magic.

4.2 What the call stack actually stores

When `preorder(2)` calls `preorder(4)`, the machine has to remember two things about the paused call:

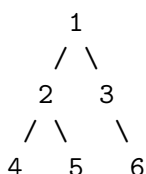
- **which node** it was working on (node 2)
- **where to resume** (it already finished the left call, so next is the right call)

That is the entire content of a stack frame for our purposes. An iterative traversal keeps exactly the same information by hand, in a `stack<TreeNode*>` plus the structure of the loop.

```
stack<TreeNode*> st;
st.push(node);    // remember to come back to this node
st.top();         // the most recently postponed node
st.pop();         // done with it
```

A stack is last in, first out. That matches recursion: the most recently postponed call is the one you resume first.

Throughout this part, the tree is:



with preorder 1 2 4 5 3 6, inorder 4 2 5 1 3 6, postorder 4 5 2 6 3 1.

4.3 Iterative preorder

This is the easy one, because preorder records a node the moment it arrives. There is nothing to postpone about the node itself, only about its children.

Algorithm: push the root. Repeatedly pop a node, record it, then push its right child and then its left child.

```
vector<int> preorderTraversal(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    stack<TreeNode*> st;
    st.push(root);

    while (!st.empty()) {
        TreeNode* node = st.top();
        st.pop();

        out.push_back(node->val);

        if (node->right) st.push(node->right); // right first
        if (node->left) st.push(node->left);  // left on top, so it pops first
    }
    return out;
}
```

4.3.1 Why right is pushed before left

A stack reverses order. We want left to be processed first, so left must be on top, so left goes in last. Swap the two lines and you get the “root, right, left” order, which is a genuinely useful traversal in its own right, as you will see in the postorder section.

4.3.2 Dry run

push 1	st = [1]	out = []
pop 1, record 1, push 3, 2	st = [3,2]	out = [1]
pop 2, record 2, push 5, 4	st = [3,5,4]	out = [1,2]
pop 4, record 4, no kids	st = [3,5]	out = [1,2,4]
pop 5, record 5, no kids	st = [3]	out = [1,2,4,5]
pop 3, record 3, push 6	st = [6]	out = [1,2,4,5,3]
pop 6, record 6	st = []	out = [1,2,4,5,3,6]

Matches the recursive output.

Time $O(N)$. **Space** $O(H)$ for the stack, which is the same as the recursion it replaces.

4.3.3 Recursive versus iterative

	Recursive	Iterative
Lines of code	4	12
Stack used	call stack	your own <code>stack<TreeNode*></code>
Deep tree risk	can overflow	safe
Readability	better	acceptable

For preorder there is no reason to prefer the iterative version except when the tree can be extremely deep, or when the interviewer asks.

4.4 Iterative inorder

Now it gets interesting. Inorder cannot record a node when it arrives, because the entire left subtree has to come out first. So arriving at a node means “postpone it and dive left”.

Algorithm:

```
curr = root
repeat:
    while curr is not empty:
        push curr
        curr = curr.left           <- dive as far left as possible
    if stack is empty: stop
    pop node, record node         <- left subtree is finished, so this node is next
    curr = node.right             <- now handle the right subtree the same way
```

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* curr = root;

    while (curr != NULL || !st.empty()) {
        while (curr != NULL) {           // go left as far as possible
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top();                  // leftmost unfinished node
        st.pop();
        out.push_back(curr->val);
        curr = curr->right;               // switch to the right subtree
    }
    return out;
}
```

4.4.1 The loop condition

`while (curr != NULL || !st.empty())` looks odd until you name the two situations:

- `curr != NULL` means “there is a subtree I have not started yet”
- `!st.empty()` means “there are nodes I postponed and still owe an answer to”

The traversal is finished only when both are false. Writing only `!st.empty()` breaks the moment the stack empties while a right subtree is still pending, which happens at node 1 in our tree.

4.4.2 Dry run

```
curr=1, st=[]
dive left: push 1, curr=2; push 2, curr=4; push 4, curr=NULL
st=[1,2,4]
pop 4, record 4, curr = 4.right = NULL      out=[4]      st=[1,2]
no dive (curr NULL)
pop 2, record 2, curr = 2.right = 5          out=[4,2]      st=[1]
dive left from 5: push 5, curr = 5.left = NULL st=[1,5]
pop 5, record 5, curr = NULL                out=[4,2,5]    st=[1]
pop 1, record 1, curr = 1.right = 3          out=[4,2,5,1]  st=[]
dive left from 3: push 3, curr = 3.left = NULL st=[3]
pop 3, record 3, curr = 3.right = 6          out=[4,2,5,1,3] st=[]
dive: push 6, curr = NULL                    st=[6]
pop 6, record 6, curr = NULL                 out=[4,2,5,1,3,6] st=[]
curr NULL and stack empty, stop
```

Time $O(N)$: each node is pushed once and popped once. **Space $O(H)$.**

4.4.3 Where this template gets reused

The BST iterator (LeetCode 173) is literally this loop, cut in half. The “dive left” part becomes a helper, and each call to `next()` performs one pop plus one dive. Kth smallest in a BST (LeetCode 230) is the same loop with a counter and an early return, which is strictly better than building the whole sorted list.

4.5 Iterative postorder

Postorder is the hardest, because a node may only be recorded after **both** subtrees are done, so when you pop a node from the stack you cannot immediately tell whether you are coming back from the left subtree or the right one.

There are three standard ways to handle this. Learn the first one properly, and know that the third exists.

4.5.1 Method 1, the reversal trick (learn this one)

Postorder is left, right, root. Reverse that sequence and you get root, right, left. And root, right, left is just preorder with the two pushes swapped, which we already know how to write.

So: do a preorder with right and left swapped, then reverse the result.

```
vector<int> postorderTraversal(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;
```

```

stack<TreeNode*> st;
st.push(root);

while (!st.empty()) {
    TreeNode* node = st.top();
    st.pop();

    out.push_back(node->val);

    if (node->left) st.push(node->left);    // note: left pushed first now
    if (node->right) st.push(node->right); // so right pops first
}

reverse(out.begin(), out.end());
return out;
}

```

4.5.1.1 Dry run

push 1	st=[1]	out=[]
pop 1, record 1, push 2, 3	st=[2,3]	out=[1]
pop 3, record 3, push 6	st=[2,6]	out=[1,3]
pop 6, record 6	st=[2]	out=[1,3,6]
pop 2, record 2, push 4, 5	st=[4,5]	out=[1,3,6,2]
pop 5, record 5	st=[4]	out=[1,3,6,2,5]
pop 4, record 4	st=[]	out=[1,3,6,2,5,4]

reverse -> [4,5,2,6,3,1]

Which is exactly the postorder.

Time $O(N)$, space $O(H)$ plus the $O(N)$ output which we were building anyway. The reversal is $O(N)$.

Downside: you do not get the nodes in postorder **as you go**, only at the end. That is fine for a traversal question and not fine if you wanted to free memory or compute values on the fly. In practice, when you actually need bottom-up computation, you use recursion, so this is rarely a problem.

4.5.2 Method 2, two stacks

The same idea written more explicitly. Push into a second stack instead of reversing at the end.

```

vector<int> postorderTwoStacks(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    stack<TreeNode*> s1, s2;
    s1.push(root);

    while (!s1.empty()) {
        TreeNode* node = s1.top(); s1.pop();
        s2.push(node);
        if (node->left) s1.push(node->left);
        if (node->right) s1.push(node->right);
    }
}

```

```

    }
    while (!s2.empty()) {
        out.push_back(s2.top()->val);
        s2.pop();
    }
    return out;
}

```

Same complexity, one extra stack. Some people find it clearer. It is the same algorithm as method 1.

4.5.3 Method 3, one stack with a lastVisited pointer

This one produces true postorder in real time, but it needs one extra idea: remember which node you most recently finished, so that when you come back up to a node you can tell whether its right subtree is already done.

```

vector<int> postorderOneStack(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    TreeNode* lastVisited = NULL;

    while (curr != NULL || !st.empty()) {
        while (curr != NULL) {           // dive left, same as inorder
            st.push(curr);
            curr = curr->left;
        }
        TreeNode* node = st.top();

        if (node->right != NULL && lastVisited != node->right) {
            curr = node->right;           // right subtree not done yet, go there
        } else {
            out.push_back(node->val);      // both subtrees done, record now
            lastVisited = node;
            st.pop();
        }
    }
    return out;
}

```

The condition `lastVisited != node->right` is the whole trick. It answers the question “have I just come back **from** my own right child?” If yes, the node is finished. If no, the right subtree still has to run.

Time $O(N)$, space $O(H)$.

Use method 1 in a contest. Use method 3 if the interviewer specifically asks for postorder without reversing, or if you must process nodes in true postorder while traversing.

4.6 Summary table

Traversal	Iterative idea	Extra state	Difficulty
Preorder	pop, record, push right then left	none	easy
Inorder	dive left pushing, pop, record, go right	none	medium
Postorder (method 1)	root right left, then reverse	none	easy
Postorder (method 3)	dive left, check if right is done	lastVisited	hard

All of them are $O(N)$ time and $O(H)$ space, exactly like the recursive versions. The iterative form does not make anything faster. It only moves the memory from the call stack to the heap and gives you control.

4.7 Common mistakes

- **Pushing left before right in iterative preorder.** You get root, right, left. Correct for the postorder trick, wrong for preorder.
- **Using `while (!st.empty())` alone in iterative inorder.** The stack is empty at the start and can be empty in the middle while work remains. You need the `curr != NULL` half too.
- **Pushing NULL onto the stack.** Guard every push with `if (node->left)`.
- **Reading `st.top()` after `st.pop()`.** Read first, pop second.
- **Trying to write postorder as “pop, record, push left, push right, no reversal”.** That is not postorder in any order. Either reverse, or use `lastVisited`.
- **Forgetting the final reverse in method 1.** You end up with root, right, left and cannot see why the answer looks “backwards”.
- **Believing the iterative version is asymptotically better.** It is not. The gain is stack safety and control.

4.8 Check yourself

1. What two pieces of information does a call stack frame hold in a traversal, and where do they live in the iterative version?
2. In iterative preorder, why must the right child be pushed before the left child?
3. Why can inorder not record a node when it first reaches it?
4. Explain the loop condition `curr != NULL || !st.empty()` in one sentence.
5. Why does reversing a “root, right, left” traversal produce postorder?
6. In method 3, what question is `lastVisited != node->right` really answering?
7. Is iterative traversal faster than recursive traversal? Justify your answer with complexities.

Part 5

Basic Tree Properties

Your first ten recursive tree functions

5.1 One template, many problems

Here is the template from Part 1 again. Every function in this part is a filled-in copy of it.

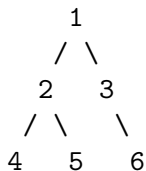
```
ReturnType solve(TreeNode* root) {  
    if (root == NULL) return answerForEmptyTree;  
  
    ReturnType L = solve(root->left);  
    ReturnType R = solve(root->right);  
  
    return combine(L, R, root->val);  
}
```

Writing a tree function is therefore a two question process:

1. **What is the answer for an empty tree?** That is your base case.
2. **If a genie handed me the correct answer for the left subtree and the correct answer for the right subtree, how would I build my answer?** That is your combine step.

Notice what you are *not* doing: you are not tracing the recursion in your head. Trust the genie. If the base case is right and the combine step is right, the recursion is right. This is the single biggest mindset shift for a beginner, and it takes a few problems before it feels natural.

Working tree for this part:



5.2 Maximum depth, also called height

LeetCode 104. Counting nodes, so a single node has depth 1.

```
int maxDepth(TreeNode* root) {  
    if (root == NULL) return 0;  
    int L = maxDepth(root->left);
```

```
int R = maxDepth(root->right);
return 1 + max(L, R);
}
```

Base case: an empty tree has no levels, so 0. Combine: my depth is the deeper of my two children, plus one for me.

5.2.1 Dry run

```
maxDepth(4) = 1 + max(0, 0) = 1
maxDepth(5) = 1 + max(0, 0) = 1
maxDepth(2) = 1 + max(1, 1) = 2
maxDepth(6) = 1 + max(0, 0) = 1
maxDepth(3) = 1 + max(0, 1) = 2
maxDepth(1) = 1 + max(2, 2) = 3
```

Answer 3. Note the order in which those lines were completed: 4, 5, 2, 6, 3, 1. That is the postorder. Every bottom-up computation happens in postorder, whether you call it that or not.

If you want the height measured in **edges** instead, the only change is the base case: return -1 for NULL. Then a single node gives $1 + \max(-1, -1) = 0$. Nothing else moves.

Time $O(N)$, **space** $O(H)$.

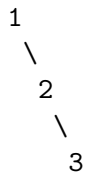
5.3 Minimum depth

LeetCode 111. The number of nodes on the shortest path from the root down to a **leaf**.

The obvious answer is wrong:

```
// WRONG
int minDepth(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + min(minDepth(root->left), minDepth(root->right));
}
```

Try it on



The left subtree of 1 is empty and returns 0, so the answer comes out as 1. But node 1 is not a leaf. The true answer is 3.

The problem is that “empty subtree” and “path ends here” are different things. A missing child does not create a leaf.

```
int minDepth(TreeNode* root) {
    if (root == NULL) return 0;
```

```
if (root->left == NULL) return 1 + minDepth(root->right);
if (root->right == NULL) return 1 + minDepth(root->left);
return 1 + min(minDepth(root->left), minDepth(root->right));
}
```

Read the middle two lines as: if I have only one child, the shortest path to a leaf must go through that child, there is no choice.

On our tree: node 3 has only a right child, so $\text{minDepth}(3) = 1 + \text{minDepth}(6) = 2$. Node 2 has both, so $\text{minDepth}(2) = 1 + \min(1, 1) = 2$. Root: $1 + \min(2, 2) = 3$.

Time $O(N)$, space $O(H)$. The BFS version from Part 3 has the same worst case but stops early in practice.

5.4 Count nodes

```
int countNodes(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

Empty tree has 0 nodes. My count is myself plus both subtrees. On our tree: 6.

Time $O(N)$, space $O(H)$.

(LeetCode 222 asks this for a *complete* tree and wants better than $O(N)$. That uses the special shape and is not needed here.)

5.5 Count leaf nodes

A leaf has **both** children NULL.

```
int countLeaves(TreeNode* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
```

Two base cases here. The NULL case protects against crashes, the leaf case is where the counting actually happens. On our tree: leaves are 4, 5, 6, so the answer is 3.

A very common wrong version writes `if (root->left == NULL || root->right == NULL) return 1;`. That would count node 3, which has one child, as a leaf. Use `&&`.

5.6 Sum of all nodes

```
int sumOfNodes(TreeNode* root) {
    if (root == NULL) return 0;
```

```
return root->val + sumOfNodes(root->left) + sumOfNodes(root->right);
}
```

Our tree: $1 + 2 + 3 + 4 + 5 + 6 = 21$.

Compare this with `countNodes`. The two functions are identical except that one adds `root->val` and the other adds 1. Recognise that and you start to see the template rather than the individual problems.

5.7 Maximum value in the tree

```
int maxValue(TreeNode* root) {
    if (root == NULL) return INT_MIN;
    return max(root->val, max(maxValue(root->left), maxValue(root->right)));
}
```

The base case deserves a moment. What is the maximum of an empty set? There is no such number, so we return the most negative value possible, which loses every `max` comparison and therefore behaves like “no contribution”. This trick, returning an identity element for the empty case, appears constantly:

Operation	Identity to return for NULL
sum	0
count	0
max	INT_MIN
min	INT_MAX
logical AND (“all nodes satisfy”)	true
logical OR (“some node satisfies”)	false
height in nodes	0
height in edges	-1

Memorise this table. It answers the base case question for most problems immediately.

5.8 Search for a value

```
bool search(TreeNode* root, int target) {
    if (root == NULL) return false;
    if (root->val == target) return true;
    return search(root->left, target) || search(root->right, target);
}
```

In a general binary tree there is no ordering, so you may have to look everywhere: $O(N)$.

C++ short circuits `||`, so if the value is found on the left, the right subtree is never visited. That is a free optimisation, but the worst case is unchanged.

If the tree is a **BST**, you can do much better by using the ordering:

```
bool searchBST(TreeNode* root, int target) {
    if (root == NULL) return false;
    if (root->val == target) return true;
    if (target < root->val) return searchBST(root->left, target);
    return searchBST(root->right, target);
}
```

Now only one side is explored, so the cost is $O(H)$: $O(\log N)$ on a balanced BST. This is the payoff for the extra ordering promise.

5.9 Same tree

LeetCode 100. Two trees are identical if they have the same shape and the same values in the same places.

New idea: the function takes **two** pointers and walks both trees in step.

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == NULL && q == NULL) return true;    // both ended together
    if (p == NULL || q == NULL) return false;   // one ended first, shapes differ
    if (p->val != q->val) return false;

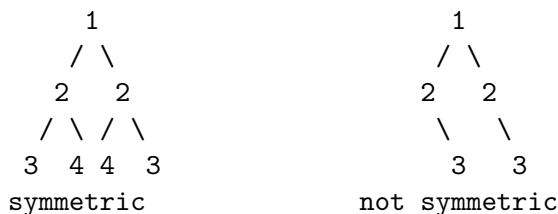
    return isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
```

The order of the first two lines matters. `&&` first, then `||`. If you write the `||` check first, you reject two empty trees, which are in fact equal.

Time $O(N)$ where N is the size of the smaller tree, since we stop at the first mismatch. **Space** $O(H)$.

5.10 Symmetric tree

LeetCode 101. Is the tree a mirror image of itself?



The mistake almost everyone makes first is to compare `root->left` with `root->right` using `isSameTree`. That checks whether the two halves are **copies**, not **mirrors**. In the symmetric picture above, the left half is $2(3,4)$ and the right half is $2(4,3)$. They are not the same tree.

The fix: write a helper that checks whether two trees are mirrors of each other, by pairing the left of one with the right of the other.

```

bool isMirror(TreeNode* a, TreeNode* b) {
    if (a == NULL && b == NULL) return true;
    if (a == NULL || b == NULL) return false;

    return a->val == b->val &&
           isMirror(a->left, b->right) &&    // outer pair
           isMirror(a->right, b->left);      // inner pair
}

bool isSymmetric(TreeNode* root) {
    if (root == NULL) return true;
    return isMirror(root->left, root->right);
}

```

Compare with `isSameTree`: the only difference is the crossing in the last two lines. Everything else is identical. Write both functions side by side once and the difference will stick.

Time $O(N)$, space $O(H)$.

5.11 The pattern behind all of this

Put the functions next to each other:

```

count      : if NULL -> 0          ; 1 + L + R
sum        : if NULL -> 0          ; val + L + R
height     : if NULL -> 0          ; 1 + max(L, R)
max value   : if NULL -> INT_MIN    ; max(val, L, R)
search     : if NULL -> false      ; (val == t) || L || R
leaves      : if NULL -> 0          ; leaf ? 1 : L + R

```

Same shape every time:

```
answer = combine(left subtree answer, right subtree answer, current node)
```

When you meet a new tree problem, do not ask “what algorithm is this”. Ask:

1. What single value would I like each subtree to hand me?
2. What is that value for an empty tree?
3. How do I merge two of them with my own node?

If you can answer those three, the code writes itself in five lines. If you cannot answer question 1 with a single number, you probably need to return a small bundle of values instead, and that is exactly what Part 6 is about.

5.12 Common mistakes

- **Missing base case.** Always the first line.
- **|| instead of && in the leaf check.** A leaf has both children NULL.
- **Assuming a missing child creates a leaf.** This is the minimum depth trap.
- **Wrong identity for the empty case.** Returning 0 for a max query breaks on all-negative trees.

- **Using `isSameTree` for symmetry.** Mirror is not the same as equal.
- **Recomputing.** Calling `maxDepth(root->left)` twice in one function doubles the work for no reason. Store it in a variable.
- **Global variables left dirty between test cases.** If you use a member variable such as `int best`, reset it at the start of the public function. On LeetCode the same object is reused across test cases and this causes bugs that only show up on the second case.

5.13 Check yourself

1. What should a height function return for NULL if you are counting edges, and why?
2. Why is the naive minimum depth solution wrong, and which tree exposes it?
3. `countNodes` and `sumOfNodes` differ by one token. Which one, and what does that tell you about the template?
4. What is the correct base value for a function that finds the minimum value in a tree?
5. Explain in one sentence why `isSymmetric` needs a two argument helper.
6. Search in a general binary tree is $O(N)$ but in a BST it is $O(H)$. What extra promise buys you that?
7. For a function that returns `true` only if every node value is even, what should the NULL case return?

Part 6

Bottom-Up Thinking

Returning information upward instead of recomputing it

6.1 The idea in one line

Solve the left subtree, solve the right subtree, combine, and hand the result to your parent.

That is all “bottom-up tree DP” means. It is the postorder position from Part 2 used deliberately. Once you can design **what** to return, you can solve diameter, balanced tree, maximum path sum, largest BST subtree, and most Medium tree problems with the same six lines.

```
solve(node)
    L = solve(node.left)
    R = solve(node.right)
    combine L, R and node
    return the result upward
```

6.2 Where the naive approach goes wrong

Here is a concrete, honest example. Suppose we want, for every node, the height of the subtree rooted at that node. The direct approach:

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

void printAllHeights(TreeNode* root) {
    if (root == NULL) return;
    cout << root->val << " has height " << height(root) << "\n";    // full walk
    printAllHeights(root->left);
    printAllHeights(root->right);
}
```

It works. But look at the cost. `printAllHeights` visits N nodes, and at each node it calls `height`, which itself walks the whole subtree below that node.

On a skewed tree of N nodes:

```
node 1: height() walks N nodes
node 2: height() walks N-1 nodes
node 3: height() walks N-2 nodes
```

...
 $\text{total} = N + (N-1) + \dots + 1 = N(N+1)/2 = O(N^2)$

On a balanced tree the same reasoning gives $O(N \log N)$. Either way, work is being repeated. When `height(1)` ran, it already computed the height of node 2 internally, then threw it away, and then we computed it again from scratch.

That throwing away is the bug. The information was already in our hands during the recursion, on the way back up.

6.3 The fix: return it instead of recomputing it

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    int L = height(root->left);    // the left subtree just told me its height
    int R = height(root->right);   // so did the right one
    cout << root->val << " has height " << 1 + max(L, R) << "\n";
    return 1 + max(L, R);
}
```

One traversal. $O(N)$. Each node computes its own answer in $O(1)$ using the two numbers its children returned.

The general principle:

If your outer function calls an inner recursive function at every node, you are almost certainly recomputing something that the recursion could have returned. Merge them into one pass.

This exact rewrite converts the naive diameter solution (Part 7) and the naive balanced check (Part 8) from $O(N^2)$ to $O(N)$. Those two problems exist on LeetCode largely to teach this rewrite.

6.4 Designing the return value

The whole skill is answering this question:

What does my parent need to know about me?

Not “what is the final answer”, but “what must travel up one edge”. Some examples:

Problem	What each node returns upward
Height	height of my subtree
Count nodes	number of nodes in my subtree
Balanced check	my height, or a flag saying I am already broken
Diameter	my height (the diameter is tracked separately)
Maximum path sum	best downward path sum starting at me
Largest BST subtree	(is my subtree a BST, min, max, size)
Sum of subtree values	sum of my subtree

Notice a recurring split in the last few rows. There are two different quantities:

- **the answer through me**, which may involve both of my children, and
- **the value I can give my parent**, which can only continue through **one** child, because a path cannot go down the left, come back, and go down the right.

Diameter and maximum path sum are the classic examples, and confusing these two is the number one error in this whole topic. We deal with it properly in Part 7.

6.5 Two ways to carry extra information

6.5.1 Way 1: return a small struct

Clean, explicit, and easy to explain in an interview.

```
struct Info {
    int height;
    int count;
};

Info solve(TreeNode* root) {
    if (root == NULL) return {0, 0};

    Info L = solve(root->left);
    Info R = solve(root->right);

    Info me;
    me.height = 1 + max(L.height, R.height);
    me.count = 1 + L.count + R.count;
    return me;
}
```

Use this when two or more values genuinely need to travel upward together.

6.5.2 Way 2: return one value, keep the answer in a reference or member

Shorter, and it is what most accepted LeetCode solutions look like.

```
class Solution {
    int best = 0; // the global answer

    int height(TreeNode* root) { // returns only the height
        if (root == NULL) return 0;
        int L = height(root->left);
        int R = height(root->right);
        best = max(best, L + R); // update the answer at this node
        return 1 + max(L, R); // hand only the height upward
    }
public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;
        height(root);
    }
}
```

```

    return best;
}
};

```

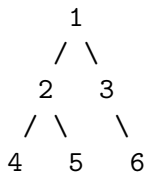
Read that carefully, because it contains the entire lesson of this part:

- $L + R$ is **the answer through this node**. It uses both children. It is recorded, not returned.
- $1 + \max(L, R)$ is **what the parent can use**. It continues through one child only.

If you ever find yourself wanting to return $L + R$ upward, stop. That would be claiming the parent can extend a path that already turned a corner, which is geometrically impossible.

Always reset the member variable at the start of the public function, since LeetCode reuses the object across test cases.

6.6 A worked example: the whole pattern on our tree



Compute, in one postorder pass, the height and node count of every subtree.

```

solve(4): L={0,0} R={0,0} -> {height 1, count 1}
solve(5): L={0,0} R={0,0} -> {height 1, count 1}
solve(2): L={1,1} R={1,1} -> {height 2, count 3}
solve(6): L={0,0} R={0,0} -> {height 1, count 1}
solve(3): L={0,0} R={1,1} -> {height 2, count 2}
solve(1): L={2,3} R={2,2} -> {height 3, count 6}

```

Six lines, six nodes, each computed exactly once. Nothing was recomputed. That is what $O(N)$ looks like.

Also notice the order the lines completed in: 4, 5, 2, 6, 3, 1, the postorder. Bottom-up and postorder are the same thing described from two directions.

6.7 When bottom-up is not the answer

Not everything flows upward. Some problems need information handed **downward** from the parent:

- “Print all root-to-leaf paths”: each child needs the path so far.
- “Path sum equals target”: each child needs the remaining target.
- “Is this a valid BST”: each child needs the allowed (min, max) range.
- “Depth of each node”: each child needs the parent’s depth.

That is a preorder shape, and it is normally written with an extra parameter:

```
void dfs(TreeNode* root, int depthSoFar) {
    if (root == NULL) return;
    dfs(root->left, depthSoFar + 1);
    dfs(root->right, depthSoFar + 1);
}
```

So the first question on any tree problem is:

Does information flow down, up, or both?

Down means preorder and extra parameters. Up means postorder and a return value. Both means you do both in the same function, which is completely normal: descend carrying context, return summaries as you unwind.

6.8 Complexity, and why it comes out to $O(N)$

For a single postorder pass:

- Each node is entered exactly once.
- At each node the combine step is $O(1)$, assuming you are combining a constant number of integers.
- Total: **$O(N)$ time.**
- Recursion depth equals the height: **$O(H)$ space**, which is $O(\log N)$ balanced and $O(N)$ skewed.

If your combine step is not $O(1)$, the total changes. For example, if at each node you merge two maps of size up to N , the cost is not $O(N)$ any more. Keep the combine cheap.

6.9 Common mistakes

- **Recomputing height inside another recursion.** The $O(N^2)$ trap.
- **Returning the “through me” value upward.** Diameter and maximum path sum both break this way, and they break silently: the answer is only wrong on some trees, so a couple of examples may still pass.
- **Forgetting to reset a member variable** between runs.
- **Choosing a return type before understanding the problem.** Decide what the parent needs first, then pick the type.
- **Using a struct when one int would do**, or a single int when the problem really needs two values. Both directions cost you time in an interview.
- **Doing the combine before the recursive calls.** At that moment you know nothing about the subtrees.

6.10 Check yourself

1. Why is calling `height()` at every node $O(N^2)$ in the worst case? Which tree shape causes it?
2. What is the difference between “the answer through this node” and “the value returned to the parent”?
3. In the diameter code above, why is `L + R` recorded but `1 + max(L, R)` returned?

4. Give a problem where information must flow downward, and say how you would carry it.
5. You want the size of the largest subtree that contains only even values. What should each node return to its parent?
6. Why is the time complexity of a single postorder pass $O(N)$ and not $O(N \log N)$?

Part 7

Diameter of a Binary Tree

The classic bottom-up problem

7.1 What diameter means

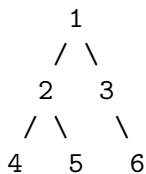
The **diameter** of a binary tree is the length of the longest path between any two nodes in the tree.

Two things people miss on first reading:

1. The path does **not** have to pass through the root.
2. The path does **not** have to start or end at a leaf, although in practice the longest one always does.

LeetCode 543 measures the length in **edges**. So a single node has diameter 0, and two nodes joined by one edge have diameter 1. We will use edges throughout. If a problem asks for the number of nodes on the path instead, the answer is simply edges + 1.

On our tree:

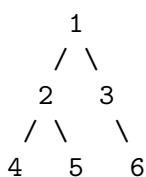


The longest path is 4 to 2 to 1 to 3 to 6, which uses 4 edges. So the diameter is 4. Another candidate is 4 to 2 to 5, which uses only 2 edges.

7.2 Height is not diameter

These get confused constantly.

- **Height** measures a path that goes straight down from one node. It never turns.
- **Diameter** measures a path that may go up from one node and then down into a different branch. It turns exactly once.

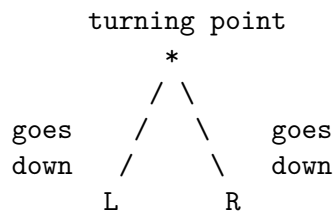


height of the tree = 2 edges (1 -> 2 -> 4)
diameter = 4 edges (4 -> 2 -> 1 -> 3 -> 6)

The diameter is at least the height and can be almost twice as much.

7.3 The key observation

Every path in a tree has a single **highest** node, the point where it turns. Call it the turning point.



Once you fix the turning point, the best possible path through it is forced: go as deep as possible into the left subtree, and as deep as possible into the right subtree.

So, if $h(x)$ is the height of the subtree rooted at x measured in **nodes** (so $h(\text{NULL}) = 0$), then

best path turning at node = $h(\text{node.left}) + h(\text{node.right})$ [in edges]

Why does that come out in edges? Going down $h(\text{left})$ nodes from node uses exactly $h(\text{left})$ edges, because the first edge takes you from node into the left subtree. Same on the right. Add them.

And then:

diameter of tree = maximum over all nodes of ($h(\text{left}) + h(\text{right})$)

Every path is counted exactly once, at its own turning point. Nothing is missed and nothing is double counted.

7.4 Brute force, and why it is slow

Translate the formula directly:

```

int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

int diameterBrute(TreeNode* root){
    if (root == NULL) return 0;

    int through = height(root->left) + height(root->right); // path turning here
    int left    = diameterBrute(root->left);                // best path entirely on the left
    int right   = diameterBrute(root->right);               // best path entirely on the right

    return max(through, max(left, right));
}
  
```

This is correct. It is also wasteful. At every node we call `height`, which walks the entire subtree below that node. We computed those heights already inside the outer recursion and threw them away.

Cost on a skewed tree:

node 1: height walk over N nodes

node 2: height walk over N-1 nodes

...

total = $O(N^2)$

For $N = 10^5$ that is 10^{10} operations. Time limit exceeded.

7.5 The $O(N)$ solution

Use Part 6. Do one postorder pass. Each call returns **only the height**, and along the way it updates a shared best answer.

```
class Solution {
    int best = 0;

    int height(TreeNode* root) {
        if (root == NULL) return 0;

        int L = height(root->left);
        int R = height(root->right);

        best = max(best, L + R);           // the path that turns HERE
        return 1 + max(L, R);             // what my parent can use
    }

public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;                         // reset, LeetCode reuses the object
        height(root);
        return best;
    }
};
```

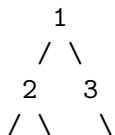
Six lines of logic. Read the two important ones again:

```
best = max(best, L + R);    // answer through this node, uses BOTH children
return 1 + max(L, R);      // value for the parent, uses ONE child
```

A path cannot go down the left branch, come back up, and then go down the right branch and still be a straight downward path for the parent. So the parent may only receive $1 + \max(L, R)$. The $L + R$ combination is real, but it is a completed path, so it is recorded and not passed on.

If you understand this distinction, you will get maximum path sum (LeetCode 124) right the first time, because it is the same shape with sums instead of counts.

7.6 Dry run



4 5 6

```
height(4): L=0 R=0   best = max(0, 0+0) = 0   return 1
height(5): L=0 R=0   best = max(0, 0)   = 0   return 1
height(2): L=1 R=1   best = max(0, 1+1) = 2   return 2
height(6): L=0 R=0   best = max(2, 0)   = 2   return 1
height(3): L=0 R=1   best = max(2, 0+1) = 2   return 2
height(1): L=2 R=2   best = max(2, 2+2) = 4   return 3
```

```
answer = 4
```

Trace the winning line. At node 1, $L = 2$ means the left subtree can reach 2 levels down (nodes 2 then 4), and $R = 2$ means the right subtree can reach 2 levels down (nodes 3 then 6). So the path is 4, 2, 1, 3, 6 with 4 edges. Exactly what we predicted from the picture.

Note also that $best$ was 2 at one point, from the path 4 to 2 to 5, and later improved. That is the algorithm considering every turning point in turn.

7.7 Complexity

Version	Time	Space
Brute force	$O(N^2)$ worst case, $O(N \log N)$ if balanced	$O(H)$
Bottom-up	$O(N)$	$O(H)$

The bottom-up version enters each node once and does $O(1)$ work there. The space is the recursion stack, so $O(H)$.

7.8 Variations you should be able to handle

- **Diameter in nodes instead of edges:** return $best + 1$, or track $L + R + 1$ inside.
- **Weighted edges:** replace $1 + \max(L, R)$ with $w + \max(L, R)$ and $L + R$ with $w_l + w_r$ style bookkeeping.
- **Maximum path sum (LeetCode 124):** same skeleton. Return $node \rightarrow val + \max(0, \max(L, R))$ upward, record $node \rightarrow val + \max(0, L) + \max(0, R)$. The $\max(0, \dots)$ is there to drop negative branches. See Part 9.
- **Longest univalue path (LeetCode 687):** same skeleton, but a child only contributes if its value equals the parent's value.

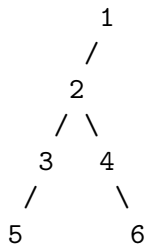
Every one of those is the same two lines with a different arithmetic. That is why diameter is worth this much attention.

7.9 Common mistakes

- **Returning $L + R$ to the parent.** This is the big one. It produces answers that are too large on some trees and correct on others, so small tests may pass.
- **Recomputing height at every node.** Correct but $O(N^2)$.
- **Mixing edges and nodes.** Decide first. With $h(\text{NULL}) = 0$ you are counting nodes for the height and $L + R$ naturally comes out in edges.
- **Forgetting to reset best.** Fails on the second test case only, which is confusing to debug.
- **Assuming the diameter passes through the root.** Build a counterexample: a root with a tiny right subtree and a long, deep left subtree whose two branches are both long.
- **Assuming the answer equals $2 * \text{height}$.** Only true for perfectly balanced shapes.

7.10 Check yourself

1. Why does every path in a tree have exactly one turning point?
2. Why is $h(\text{left}) + h(\text{right})$ a count of edges when h counts nodes?
3. What exactly is being recomputed in the brute force version?
4. What would break if `height` returned $1 + L + R$?
5. On this tree, what is the diameter?



6. State in one sentence the difference between the value recorded at a node and the value returned from it.

Part 8

Balanced Binary Tree

And the sentinel value pattern

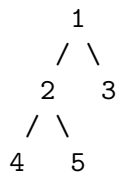
8.1 What balanced means

LeetCode 110 uses this definition:

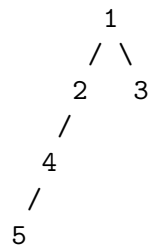
A binary tree is height balanced if, **for every node**, the heights of its left and right subtrees differ by at most 1.

The phrase “for every node” is the whole problem. It is not enough to check the root.

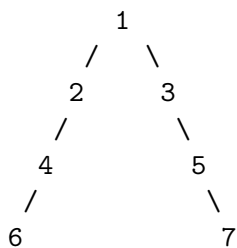
balanced



NOT balanced



In the right hand tree, look at the root: left height 3, right height 1. Difference 2. Broken. But you can also build trees where the root looks perfectly fine and a node deep inside is broken:



At the root: left height 3, right height 3, difference 0, looks fine. At node 2: left height 2, right height 0, difference 2. Not balanced. If you only check the root you will answer “yes” and be wrong.

Why do we care about this property at all? Because balance is exactly what guarantees height $O(\log N)$, which is what makes BST operations fast. AVL trees and red-black trees exist to maintain this invariant automatically.

8.2 Naive solution

Straight translation of the definition:

```

int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

bool isBalancedNaive(TreeNode* root) {
    if (root == NULL) return true;

    int L = height(root->left);
    int R = height(root->right);
    if (abs(L - R) > 1) return false;           // check me

    return isBalancedNaive(root->left) &&      // check everyone below me
           isBalancedNaive(root->right);
}

```

This is correct. It is also the same trap as the diameter brute force: at every node we call `height`, which walks the whole subtree again.

Worst case (skewed tree):

```

node 1: height walks N nodes
node 2: height walks N-1 nodes
...
total O(N^2)

```

Balanced input gives $O(N \log N)$, which is why the naive version sometimes passes small tests and dies on the big ones.

8.3 The $O(N)$ solution: the sentinel trick

We want height and balance in the same pass. There are two clean ways.

8.3.1 Way 1: return -1 to mean “already broken”

The height function normally returns a value that is 0 or larger. So the value -1 is unused, and we can hijack it to carry a second message: **this subtree is not balanced, stop caring about its height.**

```

class Solution {
    // returns the height, or -1 if this subtree is not balanced
    int check(TreeNode* root) {
        if (root == NULL) return 0;

        int L = check(root->left);
        if (L == -1) return -1;           // left is broken, propagate

        int R = check(root->right);
        if (R == -1) return -1;           // right is broken, propagate

        if (abs(L - R) > 1) return -1;     // I am broken

        return 1 + max(L, R);             // fine, report my height
    }
}

```

```

    }

public:
    bool isBalanced(TreeNode* root) {
        return check(root) != -1;
    }
};

```

Once any node reports -1, every ancestor immediately returns -1 without doing more work. The failure travels straight to the top.

Time $O(N)$ (each node visited once), **space $O(H)$** .

8.3.2 Way 2: return a small struct

If the sentinel feels like a hack, or if -1 could be a legal value in some other problem, return both facts explicitly:

```

struct Info {
    bool balanced;
    int height;
};

Info check(TreeNode* root) {
    if (root == NULL) return {true, 0};

    Info L = check(root->left);
    Info R = check(root->right);

    Info me;
    me.height = 1 + max(L.height, R.height);
    me.balanced = L.balanced && R.balanced && abs(L.height - R.height) <= 1;
    return me;
}

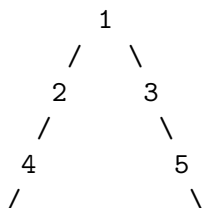
bool isBalanced(TreeNode* root) {
    return check(root).balanced;
}

```

Same complexity. More typing, zero cleverness required to read it. Both are perfectly acceptable in an interview. Say out loud which one you are doing and why.

8.4 Dry run of the sentinel version

Tree:

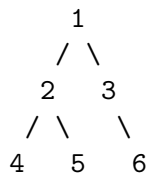


6	7		
check(6): L=0 R=0	0-0 =0 ok	return 1	
check(4): L=1 R=0	1-0 =1 ok	return 2	
check(2): L=2 R=0	2-0 =2 BAD	return -1	
check(1): L = -1	propagate	return -1	

```
isBalanced = false
```

Notice that once node 2 returned -1, the entire right subtree of the root was never even visited. The `if (L == -1) return -1;` line before computing R gives you that early exit for free. Put it there.

Now our familiar tree:



```

check(4) = 1
check(5) = 1
check(2): |1-1| = 0 ok      return 2
check(6) = 1
check(3): L=0 R=1, |0-1|=1 ok return 2
check(1): |2-2| = 0 ok      return 3

```

```
isBalanced = true
```

8.5 The pattern, stated generally

This is bigger than one problem. The pattern is:

When a recursive function must return a value **and** report a failure, and there is an impossible value in the normal range, use that impossible value as the failure signal and propagate it upward immediately.

Examples of the same idea:

- Balanced tree: height, or -1 for “not balanced”.
- Largest BST subtree: (size, min, max), or a flag saying “not a BST”.
- Checking whether all values are within a range: return the range, or an impossible range.
- Many graph problems: return a distance, or -1 for unreachable.

The one rule: **make sure the sentinel cannot occur legitimately**. If node values can be negative and you are returning sums, -1 is a terrible sentinel. Then use the struct.

8.6 Complexity summary

Version	Time	Space	Comment
Naive, height at every node	$O(N^2)$ worst case	$O(H)$	correct but slow
Sentinel -1	$O(N)$	$O(H)$	shortest accepted solution
Struct	$O(N)$	$O(H)$	clearest to explain

8.7 Common mistakes

- **Checking only the root.** The definition says every node.
- **Computing height inside isBalanced at every node.** The $O(N^2)$ trap.
- **Using -1 when -1 is a legal answer.** Then you cannot tell failure from a real value.
- **Forgetting to propagate the failure.** If a child returns -1 and you go on to compute $1 + \max(L, R)$, you produce a height of 0 or 1 and the ancestors happily report “balanced”.
- **Using $\text{abs}(L - R) >= 1$.** The condition is a difference of at most 1, so the failure test is > 1 .
- **Confusing this with “complete” or “perfect”.** Balanced is a much weaker condition. Our sample tree is balanced but neither complete nor perfect in the strict sense.

8.8 Check yourself

1. Why is checking only the root insufficient? Draw a tree that fools it.
2. What makes -1 a safe sentinel for a height function?
3. Where in the sentinel code does the early exit happen, and how much work does it save?
4. Rewrite the sentinel version’s meaning in one sentence: “check(x) returns ...”.
5. If node values could be negative and you were returning subtree sums, why would -1 be a bad sentinel, and what would you do instead?
6. Is every balanced tree complete? Is every complete tree balanced?

Part 9

Path Problems

Root-to-leaf, any-to-any, and backtracking

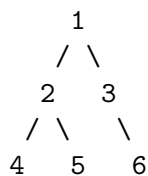
9.1 Three kinds of “path”

Problem statements use the word “path” loosely, and the three meanings need completely different code. Sort this out before you write a line.

Kind	Shape	Typical problems	Technique
Root to leaf	starts at the root, ends at a leaf	112, 113, 129, 257	preorder, carry state down
Downward path	starts anywhere, goes only down	437 Path Sum III	preorder plus prefix sums
Any to any	goes up and then down, turns once	124 Max Path Sum, 543 Diameter	postorder, bottom-up

The signature question: **can the path turn a corner?** If yes, it is a bottom-up problem and you need the “record versus return” split from Parts 6 and 7. If no, it is a top-down problem and you carry information down as a parameter.

Working tree:



Its root to leaf paths are 1-2-4, 1-2-5, and 1-3-6.

9.2 Root to leaf: the basic shape

Information flows **down**. Each child needs to know what happened above it. So you pass an extra parameter, and the recursion is preorder.

```
void dfs(TreeNode* root, int sumSoFar) {  
    if (root == NULL) return;  
  
    sumSoFar += root->val;
```

```

if (root->left == NULL && root->right == NULL) {
    // this is a leaf, sumSoFar is a complete root-to-leaf sum
    return;
}

dfs(root->left, sumSoFar);
dfs(root->right, sumSoFar);
}

```

Two things to be careful about:

- The leaf test is `left == NULL && right == NULL`. Both. Always.
- `sumSoFar` is passed **by value**, so each branch automatically gets its own copy. There is nothing to undo. When you pass by value, the “backtracking” happens for free.

9.2.1 Path Sum (LeetCode 112)

Does some root to leaf path add up to target?

```

bool hasPathSum(TreeNode* root, int target) {
    if (root == NULL) return false;

    target -= root->val;

    if (root->left == NULL && root->right == NULL)
        return target == 0;

    return hasPathSum(root->left, target) || hasPathSum(root->right, target);
}

```

Instead of accumulating a sum, we subtract as we descend, and ask whether the remaining target is exactly 0 at a leaf. Same thing, one variable fewer.

Trace on our tree with target 7: $7 - 1 = 6$ at node 1; $6 - 2 = 4$ at node 2; $4 - 4 = 0$ at leaf 4, so true.

Watch the base case. `if (root == NULL) return target == 0;` is **wrong**. Consider a root with value 5, target 5, and only a left child. Node 5’s right child is NULL, and NULL would report success even though we never reached a leaf. The leaf test must be done at the leaf, not at NULL.

Time $O(N)$, **space** $O(H)$.

9.2.2 Maximum root to leaf sum

```

int maxRootToLeaf(TreeNode* root) {
    if (root == NULL) return INT_MIN;           // no path exists here
    if (root->left == NULL && root->right == NULL) return root->val;

    return root->val + max(maxRootToLeaf(root->left), maxRootToLeaf(root->right));
}

```

This one is written bottom-up instead, and it is worth comparing with the previous two. Both directions work for root to leaf problems. Choose the one that makes the code shorter. Here `INT_MIN` for NULL means “this branch offers nothing”, so it always loses the max. Careful: if the tree can have very negative values, `root->val + INT_MIN` overflows. Guard the single child case explicitly, or use

long long.

9.3 Collecting the actual paths: backtracking

LeetCode 257 asks for all root to leaf paths, printed. Now we need the path itself, not just a number. Storing a `vector<int>` by value at every call would be $O(N)$ copying per node. Instead keep **one** shared vector and undo your change on the way out.

```
void dfs(TreeNode* root, vector<int>& path, vector<vector<int>>& res) {
    if (root == NULL) return;

    path.push_back(root->val);                // do

    if (root->left == NULL && root->right == NULL) {
        res.push_back(path);                 // record a copy
    } else {
        dfs(root->left, path, res);
        dfs(root->right, path, res);
    }

    path.pop_back();                          // undo
}
```

The push_back and pop_back must pair up. That pairing is what backtracking means: the shared state is exactly correct when you enter a node and exactly restored when you leave it.

9.3.1 Dry run on our tree

```
enter 1  path=[1]
  enter 2  path=[1,2]
    enter 4  path=[1,2,4]  leaf -> record [1,2,4]
    leave 4  path=[1,2]
    enter 5  path=[1,2,5]  leaf -> record [1,2,5]
    leave 5  path=[1,2]
  leave 2  path=[1]
  enter 3  path=[1,3]
    left is NULL, returns immediately
    enter 6  path=[1,3,6]  leaf -> record [1,3,6]
    leave 6  path=[1,3]
  leave 3  path=[1]
leave 1  path=[]
```

Result: `[[1,2,4], [1,2,5], [1,3,6]]`.

Delete the `pop_back` and the second path comes out as `[1,2,4,5]`. That is the classic symptom: paths that grow and never shrink.

Time $O(N * H)$ because copying each recorded path costs $O(H)$ and there can be $O(N)$ leaves. **Space $O(H)$** for the path plus the output.

9.3.2 Path Sum II (LeetCode 113)

Combine the two ideas above: carry the remaining target down, and keep the path with push and pop. Record path when you are at a leaf and the remaining target is 0. Write it yourself before looking at any editorial. It is the single best exercise in this part.

9.3.3 Sum Root to Leaf Numbers (LeetCode 129)

Each root to leaf path spells a number, for example 1-2-4 means 124. Carry the number down:

```
int dfs(TreeNode* root, int cur) {
    if (root == NULL) return 0;

    cur = cur * 10 + root->val;

    if (root->left == NULL && root->right == NULL) return cur;

    return dfs(root->left, cur) + dfs(root->right, cur);
}
// call dfs(root, 0)
```

Here NULL returning 0 is safe, because a missing child simply contributes nothing to the sum.

9.4 Path from the root to a specific node

Useful on its own and as one way to solve LCA.

```
bool findPath(TreeNode* root, int target, vector<int>& path) {
    if (root == NULL) return false;

    path.push_back(root->val);

    if (root->val == target) return true;           // found, keep the path

    if (findPath(root->left, target, path)) return true;
    if (findPath(root->right, target, path)) return true;

    path.pop_back();                               // not on this branch, undo
    return false;
}
```

Note the asymmetry: when we succeed we deliberately do **not** pop, because the caller wants the path preserved. Every return true skips the undo. That is intentional, and it is worth staring at for a moment.

9.5 Any to any: maximum path sum

LeetCode 124. The path may start and end anywhere, and it turns at most once. This is the diameter skeleton with sums.

```

class Solution {
    int best;

    int gain(TreeNode* root) {
        if (root == NULL) return 0;

        int L = max(0, gain(root->left));    // a negative branch is worse than nothing
        int R = max(0, gain(root->right));

        best = max(best, root->val + L + R);  // path turning HERE, uses both sides

        return root->val + max(L, R);        // for my parent, one side only
    }

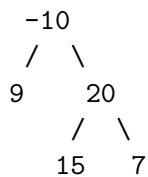
public:
    int maxPathSum(TreeNode* root) {
        best = INT_MIN;
        gain(root);
        return best;
    }
};

```

Three details, all of which are commonly missed:

1. **max(0, ...)**: if a subtree's best downward sum is negative, do not attach it. Take nothing instead. This is only legal because a path is allowed to stop at the current node.
2. **best starts at INT_MIN**, not 0. If every value is negative, the answer is the single largest (least negative) node, and starting at 0 would wrongly return 0.
3. **Record val + L + R, return val + max(L, R)**. Exactly the diameter split. Returning val + L + R upward would let the parent extend a path that already turned, which is not a path.

9.5.1 Dry run



```

gain(9)  = 9,    best = max(INT_MIN, 9)          = 9
gain(15) = 15,   best = max(9, 15)               = 15
gain(7)  = 7,    best = max(15, 7)               = 15
gain(20): L=15 R=7  best = max(15, 20+15+7)      = 42  return 20 + 15 = 35
gain(-10): L=9 R=35 best = max(42, -10+9+35)     = 42  return -10 + 35 = 25

```

answer = 42 (the path 15 -> 20 -> 7)

9.6 Downward path starting anywhere

LeetCode 437, Path Sum III: count paths that go downward but need not start at the root or end at a leaf.

The beginner solution: for every node, run a “count downward paths starting here” DFS. That is $O(N^2)$, and it passes on LeetCode but is not the intended answer.

The good solution reuses a classic array trick. Keep the running sum from the root, and a hash map from prefix sum to how many times it has occurred on the current root-to-node path. At a node with running sum S , the number of valid paths ending here is $\text{count}[S - \text{target}]$. Add your prefix sum to the map on the way in, remove it on the way out. That undo is the same backtracking discipline as above. **$O(N)$ time, $O(H)$ space.**

You do not need to master this on day one. Recognise it as “prefix sum on a tree path” and come back to it after you are comfortable with the rest.

9.7 Common mistakes

- **Treating a node with one child as a leaf.** Both children must be NULL.
- **Doing the target check at NULL instead of at a leaf.**
- **Forgetting pop_back.** Paths keep growing.
- **Popping after a successful find in findPath.** That destroys the answer you just built.
- **Passing the path by value everywhere.** It is correct but copies $O(H)$ at every node.
- **Starting best at 0 in maximum path sum.** Fails on all-negative trees.
- **Returning val + L + R upward.** The parent cannot use a path that already turned.
- **Forgetting max(0, ...)** and dragging negative subtrees into the answer.
- **Overflow.** Sums of 10^4 nodes with values up to 10^4 fit in `int`, but when in doubt use `long`.

9.8 Check yourself

1. Which three kinds of path are there, and what single question separates them?
2. Why does passing `sumSoFar` by value need no explicit backtracking?
3. Why is `if (root == NULL) return target == 0;` wrong in Path Sum?
4. In `findPath`, why is there no `pop_back` on the successful return?
5. In maximum path sum, why is `max(0, gain(child))` used, and what would break without it?
6. Why must `best` start at `INT_MIN` and not 0 in problem 124?
7. What is the time complexity of collecting all root to leaf paths, and where does the extra factor come from?

Part 10

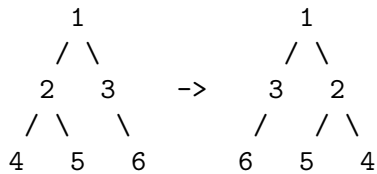
Core LeetCode Patterns

Invert, subtree, LCA, construction, serialization, and just enough BST

These are the remaining patterns that appear over and over in Easy and Medium tree problems. Each one is short. What matters is recognising which one a new problem belongs to.

10.1 Invert a binary tree

LeetCode 226. Mirror the tree left to right.



```
TreeNode* invertTree(TreeNode* root) {
    if (root == NULL) return NULL;

    TreeNode* L = invertTree(root->left);
    TreeNode* R = invertTree(root->right);

    root->left = R;
    root->right = L;
    return root;
}
```

Read it as the standard template where the return value is “the fixed up version of my subtree”. Invert both sides, then swap.

You can also write it as: swap first, then recurse. Both work, because swapping and inverting are independent operations. What does **not** work is swapping and then recursing into `root->left` twice, which is easy to do by accident after the swap.

Time $O(N)$, **space** $O(H)$. A BFS version with a queue works equally well and is stack safe.

10.2 Merge two binary trees

LeetCode 617. Overlay two trees, adding values where both have a node.

```
TreeNode* mergeTrees(TreeNode* a, TreeNode* b) {
    if (a == NULL) return b;
```

```

    if (b == NULL) return a;

    a->val += b->val;
    a->left = mergeTrees(a->left, b->left);
    a->right = mergeTrees(a->right, b->right);
    return a;
}

```

The two base cases carry all the logic: if one side is missing, the answer is simply the other side, whole. This “walk two trees in step” shape is the same one used for Same Tree and Symmetric Tree in Part 5.

10.3 Subtree of another tree

LeetCode 572. Does root contain a subtree identical to subRoot?

Reuse `isSameTree` from Part 5, then try every node as a starting point.

```

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == NULL && q == NULL) return true;
    if (p == NULL || q == NULL) return false;
    return p->val == q->val &&
           isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}

bool isSubtree(TreeNode* root, TreeNode* subRoot) {
    if (root == NULL) return subRoot == NULL;
    if (isSameTree(root, subRoot)) return true;
    return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
}

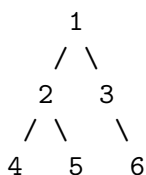
```

Time $O(N * M)$ in the worst case, where N and M are the two sizes. That is accepted for the given constraints. This is one of the rare cases where the “call a helper at every node” shape is the intended solution, because the helper stops early at the first mismatch.

Important distinction: a **subtree** must include a node and **all** of its descendants. A shape that appears in the middle of the tree but has extra children below it does not count. Students often confuse this with subtree matching in string problems.

10.4 Lowest Common Ancestor

LeetCode 236. Given two nodes p and q , find the deepest node that has both of them as descendants. A node counts as a descendant of itself.



$LCA(4, 5) = 2$
 $LCA(4, 6) = 1$
 $LCA(2, 4) = 2$ (a node can be its own ancestor)

10.4.1 The standard solution

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (root == NULL) return NULL;
    if (root == p || root == q) return root;

    TreeNode* L = lowestCommonAncestor(root->left, p, q);
    TreeNode* R = lowestCommonAncestor(root->right, p, q);

    if (L != NULL && R != NULL) return root;    // p and q are on opposite sides: I am the LCA
    return (L != NULL) ? L : R;                // both on one side, or neither found
}

```

The code is five lines and completely opaque until you state what the return value **means**:

$f(x)$ returns the LCA if both p and q are inside x 's subtree; otherwise it returns whichever of p or q it found; otherwise `NULL`.

With that sentence, every line is obvious:

- If I am p or q , I report myself and stop digging. (Correct even if the other node is below me, since then I am the LCA.)
- If the left returned something and the right returned something, p and q are in different subtrees, so I am the meeting point.
- Otherwise pass up whatever was found.

This assumes both p and q actually exist in the tree, which LeetCode 236 guarantees. If they might not exist, you need extra flags to distinguish “found both” from “found one”.

Time $O(N)$, space $O(H)$.

10.4.2 LCA in a BST

LeetCode 235. With the ordering promise you never have to search both sides.

```

TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)    root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;                               // they split here
    }
    return NULL;
}

```

The first node where p and q fall on opposite sides (or where one equals the node) is the LCA. **$O(H)$ time, $O(1)$ space.**

10.4.3 The path based alternative

Find the root to p path and the root to q path using `findPath` from Part 9, then walk both from the front while they agree. The last agreeing node is the LCA. It is $O(N)$ time and $O(H)$ space, easier

to explain, and it generalises to “distance between two nodes” (walk both paths, add the leftover lengths).

10.5 Build a tree from traversal arrays

10.5.1 Preorder plus inorder

LeetCode 105. The two facts you need:

- The **first** element of preorder is the root.
- Finding that value in inorder splits inorder into “everything on the left” and “everything on the right”.

preorder: [1, 2, 4, 5, 3, 6]

inorder : [4, 2, 5, 1, 3, 6]

root = 1

inorder left = [4, 2, 5] -> 3 nodes

inorder right = [3, 6]

preorder left = next 3 elements = [2, 4, 5]

preorder right= [3, 6]

Then recurse on each half. Searching for the root inside inorder every time costs $O(N)$ each, giving $O(N^2)$, so precompute a map from value to index.

```
class Solution {
    unordered_map<int,int> pos;    // value -> index in inorder
    int preIdx = 0;

    TreeNode* build(vector<int>& preorder, int inL, int inR) {
        if (inL > inR) return NULL;

        int val = preorder[preIdx++];
        TreeNode* root = new TreeNode(val);
        int mid = pos[val];

        root->left = build(preorder, inL, mid - 1);
        root->right = build(preorder, mid + 1, inR);
        return root;
    }

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        pos.clear();
        for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
        preIdx = 0;
        return build(preorder, 0, (int)inorder.size() - 1);
    }
};
```

The single global preIdx works because the left subtree is built completely before the right subtree starts, so preorder is consumed in exactly the right order. Build the left before the right, or everything

breaks.

Time $O(N)$, space $O(N)$ for the map plus $O(H)$ recursion. This assumes distinct values, which the problem guarantees.

10.5.2 Postorder plus inorder

LeetCode 106. Same idea with two changes: the root is the **last** element of postorder, so consume postorder from the back, and build the **right** subtree before the left.

10.5.3 Why inorder alone is not enough

Inorder [4, 2, 5] is produced by several different trees. Without knowing which node is the root you cannot split. Preorder alone is also insufficient in general. You need one order that identifies the root plus inorder that splits the rest. (Preorder plus postorder is also ambiguous for general binary trees.)

10.6 Serialize and deserialize

LeetCode 297. Turn a tree into a string and back.

The intuition: a traversal alone loses the shape, because you cannot tell missing children from present ones. Fix that by writing an explicit marker for every NULL. Preorder plus null markers determines the tree uniquely.

```
void ser(TreeNode* root, string& s) {
    if (root == NULL) { s += "#,"; return; }
    s += to_string(root->val) + ",";
    ser(root->left, s);
    ser(root->right, s);
}
```

Our tree becomes 1,2,4,#,#,5,#,#,3,#,6,#,#,.

Rebuilding reads the tokens in the same preorder:

```
TreeNode* des(vector<string>& tok, int& i) {
    if (tok[i] == "#") { i++; return NULL; }
    TreeNode* root = new TreeNode(stoi(tok[i++]));
    root->left = des(tok, i);
    root->right = des(tok, i);
    return root;
}
```

`int& i` is passed by reference so all calls share one cursor, exactly like `preIdx` above. This is the same pattern twice, and it is worth naming: **one shared cursor walking a linear sequence while the recursion rebuilds the structure.**

Time $O(N)$, space $O(N)$. A BFS version with null markers works too and is what LeetCode's own display format uses.

10.7 Just enough BST

You do not need a full BST course to handle Easy and Medium tree problems, but these five come up often.

10.7.1 Search

$O(H)$ instead of $O(N)$, because the ordering tells you which side to enter. See Part 5.

10.7.2 Insert

LeetCode 701.

```
TreeNode* insertIntoBST(TreeNode* root, int val) {  
    if (root == NULL) return new TreeNode(val);  
    if (val < root->val) root->left = insertIntoBST(root->left, val);  
    else root->right = insertIntoBST(root->right, val);  
    return root;  
}
```

The `root->left = f(root->left)` idiom is worth noticing. It handles “create a new node here” and “nothing changed” with the same line. Deletion in a BST uses the same idiom and is genuinely fiddly, so leave it for later.

10.7.3 Validate a BST

LeetCode 98. The local check `left->val < root->val < right->val` is wrong, as shown in Part 1. Two correct approaches:

Range approach, information flowing down:

```
bool valid(TreeNode* root, long long lo, long long hi) {  
    if (root == NULL) return true;  
    if (root->val <= lo || root->val >= hi) return false;  
    return valid(root->left, lo, root->val) &&  
           valid(root->right, root->val, hi);  
}  
  
// call valid(root, LLONG_MIN, LLONG_MAX)
```

Every node narrows the allowed range for its children. Use `long long` bounds so that a node holding `INT_MIN` or `INT_MAX` does not break the comparison.

Inorder approach, using the sorted property: walk inorder and check that each value is strictly greater than the previous one. Keep a `TreeNode* prev` outside the recursion.

10.7.4 Kth smallest

LeetCode 230. Inorder with a counter, returning as soon as the counter hits k . $O(H + k)$, better than building the whole list.

10.7.5 Sorted array to balanced BST

LeetCode 108. Pick the middle element as the root, recurse on both halves. The middle choice is what guarantees balance.

```
TreeNode* build(vector<int>& a, int l, int r) {
    if (l > r) return NULL;
    int mid = l + (r - l) / 2;
    TreeNode* root = new TreeNode(a[mid]);
    root->left = build(a, l, mid - 1);
    root->right = build(a, mid + 1, r);
    return root;
}
```

Time $O(N)$, space $O(\log N)$.

10.8 Common mistakes

- **Recurring into `root->left` after already swapping it in invert.**
- **Confusing subtree with subgraph.** A subtree carries all descendants.
- **Explaining LCA code without stating what the function returns.** In an interview, state the meaning first, then the code.
- **Building the right subtree before the left in problem 105.** The shared preorder cursor makes order mandatory.
- **Searching inorder linearly for the root.** $O(N^2)$. Use a map.
- **Serializing without null markers.** The shape is lost and cannot be recovered.
- **Passing the cursor by value in deserialize.** Every call gets its own copy and the tree comes out wrong.
- **Using `INT_MIN` and `INT_MAX` as the initial range in validate BST.** Fails when a node actually holds those values. Use `long long`.
- **Assuming the tree is a BST when the problem never said so.**

10.9 Check yourself

1. In one sentence, what does the LCA function return for an arbitrary node?
2. Why does building the tree from preorder and inorder require a map, and what is the complexity without it?
3. Why is inorder alone not enough to rebuild a tree?
4. Why must the deserialize cursor be shared across calls?
5. What is wrong with checking only `left->val < root->val < right->val` for a BST?
6. Why does choosing the middle element in problem 108 give a balanced tree?
7. Subtree of another tree runs in $O(N \cdot M)$. Why is that acceptable here, whereas an $O(N^2)$ diameter solution is not?

Part 11

Tree Views

Horizontal distance, top, bottom, vertical, boundary, left and right

11.1 The idea: give every node a coordinate

So far a node had one coordinate, its level. For view problems we add a second one.

Horizontal distance (HD), sometimes called the column:

$HD(\text{root}) = 0$

$HD(\text{left child}) = HD(\text{parent}) - 1$

$HD(\text{right child}) = HD(\text{parent}) + 1$

Level is the vertical coordinate, HD is the horizontal one. Together they place every node on a grid, and every view question becomes a question about that grid.

Our tree with both coordinates:

	hd=-2	hd=-1	hd=0	hd=1	hd=2
level 0			1		
level 1		2		3	
level 2	4		5		6
node : 1	2	3	4	5	6
hd : 0	-1	+1	-2	0	+2
level: 0	1	1	2	2	2

Note that node 1 and node 5 share the column $hd = 0$. That collision is the interesting part of every view problem.

Once the grid is in your head:

View	Question about the grid
Top view	in each column, the node with the smallest level
Bottom view	in each column, the node with the largest level
Vertical order	in each column, all nodes, top to bottom
Left view	in each row, the leftmost node
Right view	in each row, the rightmost node
Boundary	walk the outline of the drawing

11.2 Why BFS is the natural tool

For top view we need “the first node seen in each column”. If you use DFS you might reach column 0 through node 5 at level 2 before ever reaching some other column, and worse, DFS explores the whole left subtree before the right, so it can enter a column at a deep level before a shallower node in the same column is discovered.

BFS visits nodes in increasing order of level, so the **first** time BFS touches a column, that is the topmost node of the column. That single property makes top view a three line change to the BFS template.

If you insist on DFS, you must carry (hd, level) and keep the node with the minimum level per column. That works and is sometimes shorter, but BFS is the version to learn first.

11.3 Top view

The nodes you would see looking down at the tree from above.

```
vector<int> topView(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;

    map<int,int> col;                                // hd -> value, kept sorted by hd
    queue<pair<TreeNode*,int>> q;                    // node with its hd
    q.push({root, 0});

    while (!q.empty()) {
        auto cur = q.front(); q.pop();
        TreeNode* node = cur.first;
        int hd = cur.second;

        if (col.find(hd) == col.end())                // first node in this column wins
            col[hd] = node->val;

        if (node->left) q.push({node->left, hd - 1});
        if (node->right) q.push({node->right, hd + 1});
    }

    for (auto& p : col) res.push_back(p.second);    // map iterates in increasing hd
    return res;
}
```

11.3.1 Dry run

q = [(1,0)]		
pop (1, 0)	col empty at 0 -> col[0]=1	push (2,-1), (3,1)
pop (2,-1)	col[-1]=2	push (4,-2), (5,0)
pop (3, 1)	col[1]=3	push (6,2)
pop (4,-2)	col[-2]=4	
pop (5, 0)	column 0 already has 1, skip	

```
pop (6, 2)    col[2]=6
```

```
col = { -2:4, -1:2, 0:1, 1:3, 2:6 }
top view = 4 2 1 3 6
```

Node 5 is hidden behind node 1. That is exactly what “looking from above” means.

Time $O(N \log N)$ because of the map. **Space** $O(N)$.

11.4 Bottom view

Same code, one word different: always overwrite.

```
col[hd] = node->val;    // last node in this column wins
```

Since BFS processes levels in order, the last node written for a column is the deepest one.

```
col = { -2:4, -1:2, 0:5, 1:3, 2:6 }
bottom view = 4 2 5 3 6
```

Column 0 now shows 5 instead of 1. Everything else is unchanged.

Careful with ties. If two nodes share the same column **and** the same level, which one is “the bottom”? The usual convention, and what BFS gives you naturally, is the one that appears later in left to right order. Different sources define this differently, so if a judge disagrees with you, this is the line to look at.

11.5 Vertical order traversal

Now keep everything in each column instead of just one node.

```
vector<vector<int>> verticalOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (root == NULL) return res;

    map<int, vector<int>> col;
    queue<pair<TreeNode*,int>> q;
    q.push({root, 0});

    while (!q.empty()) {
        auto cur = q.front(); q.pop();
        TreeNode* node = cur.first;
        int hd = cur.second;

        col[hd].push_back(node->val);

        if (node->left) q.push({node->left, hd - 1});
        if (node->right) q.push({node->right, hd + 1});
    }
}
```

```

    for (auto& p : col) res.push_back(p.second);
    return res;
}

```

For our tree: `[[4], [2], [1,5], [3], [6]]`.

Time $O(N \log N)$, space $O(N)$.

11.5.1 The LeetCode 987 twist

LeetCode 987 adds a tie-breaking rule: within a column, sort by row first, and if two nodes share the same row and column, sort by **value**. Plain BFS does not guarantee that, so the standard solution collects triples `(col, row, value)`, sorts them, and then groups by column.

```

// collect with DFS or BFS into vector<array<int,3>> v of (col, row, val)
sort(v.begin(), v.end());
// then group consecutive entries with equal col

```

Read the problem statement carefully. LeetCode 314 (vertical order, premium) does not have the value tie-break, LeetCode 987 does. This is a place where copying a remembered solution without reading gets you a wrong answer.

11.6 Right view and left view

Covered in Part 3 with BFS: the last node of each level is the right view, the first is the left view. For our tree, right view is 1 3 6 and left view is 1 2 4.

The DFS version is worth knowing because it is shorter:

```

void dfs(TreeNode* root, int level, vector<int>& res) {
    if (root == NULL) return;

    if (level == (int)res.size()) // first node reaching this level
        res.push_back(root->val);

    dfs(root->right, level + 1, res); // right first for the RIGHT view
    dfs(root->left, level + 1, res);
}

```

Visit right before left, and the first node that reaches each new level is the rightmost one. Swap the two calls for the left view. The condition `level == res.size()` is a neat way of asking “is this level new?” without a map.

11.7 Boundary traversal

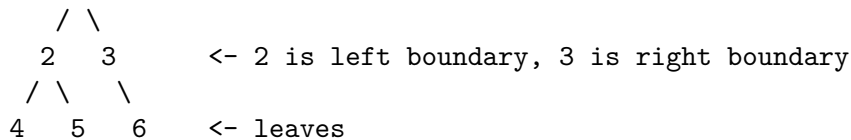
Print the outline of the tree, anticlockwise, starting at the root: left boundary going down, then all leaves left to right, then the right boundary coming up. Each node appears exactly once.

For our tree the answer is 1 2 4 5 6 3.

```

1          <- root

```



Three pieces, and the whole difficulty is avoiding duplicates where they meet.

```

bool isLeaf(TreeNode* n) {
    return n->left == NULL && n->right == NULL;
}

void addLeftBoundary(TreeNode* root, vector<int>& res) {
    TreeNode* cur = root->left;
    while (cur != NULL) {
        if (!isLeaf(cur)) res.push_back(cur->val);    // leaves handled separately
        if (cur->left) cur = cur->left;
        else cur = cur->right;                        // no left child, go right
    }
}

void addLeaves(TreeNode* root, vector<int>& res) {
    if (isLeaf(root)) { res.push_back(root->val); return; }
    if (root->left) addLeaves(root->left, res);
    if (root->right) addLeaves(root->right, res);
}

void addRightBoundary(TreeNode* root, vector<int>& res) {
    TreeNode* cur = root->right;
    vector<int> tmp;
    while (cur != NULL) {
        if (!isLeaf(cur)) tmp.push_back(cur->val);
        if (cur->right) cur = cur->right;
        else cur = cur->left;
    }
    reverse(tmp.begin(), tmp.end());                // bottom to top
    for (int v : tmp) res.push_back(v);
}

vector<int> boundaryTraversal(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;

    if (!isLeaf(root)) res.push_back(root->val);    // a single node is a leaf, do not double count

    addLeftBoundary(root, res);
    addLeaves(root, res);
    addRightBoundary(root, res);
    return res;
}

```

11.7.1 The three traps, all about duplicates

1. **The root.** If the tree is a single node, the root is also a leaf. Adding it in both places prints it twice. Hence the `if (!isLeaf(root))` guard.

2. **Leaves on the boundaries.** Node 4 is both on the left boundary and a leaf. The boundary walkers skip leaves, and `addLeaves` handles all of them, in order.
3. **The right boundary direction.** It must come out bottom to top, so collect it into a temporary vector and reverse. Recursion can do this too, by recursing before pushing.

Also note the “if there is no left child, go right” rule inside the boundary walk. The left boundary is not “keep going left”. It is the leftmost path you can trace, which may bend right when it runs out of left children.

Time $O(N)$, space $O(H)$. The leaves walk is $O(N)$, the two boundary walks are $O(H)$ each.

11.8 Choosing the data structure

Need	Use	Why
Group by column, output sorted by column	<code>map<int, ...></code>	ordered iteration, no sorting step
Only one node per column	<code>map<int,int></code>	first wins for top, last wins for bottom
All nodes per column	<code>map<int, vector<int>></code>	preserves BFS order inside a column
Node plus its column	<code>queue<pair<TreeNode*,int>></code>	carry the coordinate along with the node
LeetCode 987 tie-breaking	vector of triples plus sort	needs a custom order that a map cannot express
Level based views	plain BFS	no map needed at all

You will see `multimap` mentioned in some references. You rarely need it. `map<int, vector<int>>` is easier to read and does the same job.

If the $O(\log N)$ factor bothers you, note that HD values lie in the range $[-N, N]$, so you can find the minimum and maximum HD in a first pass and use a vector with an offset instead of a map, giving $O(N)$. This is almost never needed on LeetCode, but it is a good answer if an interviewer asks how to remove the log factor.

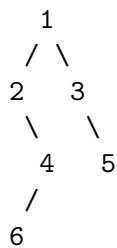
11.9 Common mistakes

- **Using DFS for top or bottom view without tracking the level.** DFS can reach a column at a deep level first and record the wrong node.
- **Using `unordered_map` for columns.** The output must be ordered by column, and `unordered_map` is not. You would have to sort the keys anyway.
- **Updating HD with `hd + 1` on the left child.** Left is minus one. Write the two lines carefully.
- **Forgetting the empty tree.**
- **Boundary traversal printing the root twice** when the root is a leaf.
- **Boundary traversal printing leaves twice**, once as part of a boundary and once from the leaves pass.

- **Not reversing the right boundary.**
- **Believing the left boundary is “always go left”.** When there is no left child you follow the right child.
- **Ignoring LeetCode 987’s value tie-break.**

11.10 Check yourself

1. What are the HD values of every node in our sample tree, and which nodes collide?
2. Why does BFS give the top view for free, while DFS needs extra bookkeeping?
3. What is the only change between the top view and the bottom view code?
4. Why is `map` preferred over `unordered_map` here?
5. In boundary traversal, why do the boundary walkers deliberately skip leaves?
6. Why is the right boundary collected and then reversed?
7. Give the top view, bottom view, and boundary traversal of this tree:



Part 12

Morris Traversal

Inorder in $O(1)$ extra space

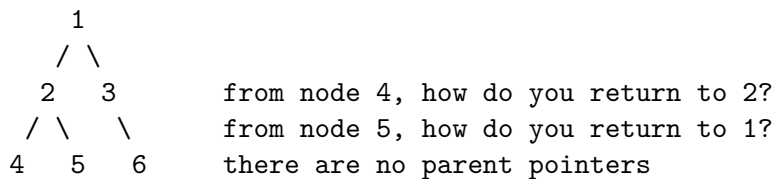
Read this part only after iterative inorder (Part 4) feels comfortable. Morris traversal is the one advanced item in this course. You do not need it for most problems, but it is a favourite interview question and it teaches something real about what a stack is for.

12.1 The question it answers

Both inorder implementations we know use $O(H)$ extra memory:

- recursion uses the call stack
- the iterative version uses an explicit `stack<TreeNode*>`

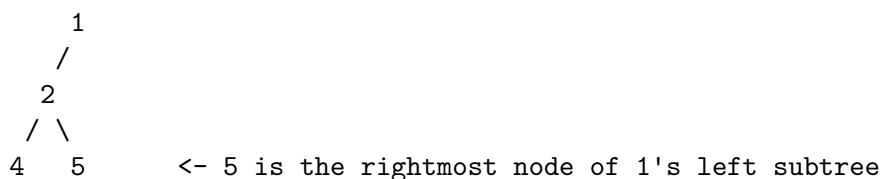
Why is that memory needed at all? Because of one specific moment. You are at node 1, you dive into the left subtree, and when the left subtree finishes you have to **get back to node 1**. Nothing inside the left subtree points upward. Without a stack, once you go down you cannot come back.



Morris traversal's answer: **build the return path into the tree itself, temporarily, and remove it afterwards.**

12.2 The key observation

Take node 1. The last node visited in its left subtree, in inorder, is node 5. In general, the last node of a left subtree in inorder is its **rightmost** node, reached by going left once and then right as far as possible. That node is called the **inorder predecessor** of node 1.



Here is the point: node 5 has an empty right pointer. That pointer is currently wasted. So before diving into the left subtree, set

```
5->right = 1
```

Now, when the traversal walks off the end of the left subtree, it lands on node 1 automatically. No stack needed. A pointer used this way is called a **thread**, which is why this is sometimes called threaded tree traversal.

Of course, we must remove the thread later, or we have created a cycle and left the tree corrupted.

12.3 The algorithm

```
curr = root
while curr is not NULL:

    if curr has no left child:
        visit curr
        curr = curr.right           // may be a real edge or a thread

    else:
        pred = rightmost node of curr's left subtree
            (go left once, then right while right exists and right != curr)

        if pred.right == NULL:      // first time here
            pred.right = curr        // create the thread
            curr = curr.left         // go explore the left subtree

        else:                       // pred.right == curr, second time here
            pred.right = NULL        // remove the thread, restore the tree
            visit curr               // left subtree is finished, so now visit
            curr = curr.right        // move on to the right subtree
```

The `pred.right != curr` condition inside the search is essential. Without it, once a thread exists the search loops forever along it.

The whole design turns on one fact: **you arrive at every node with a left child exactly twice**. The first arrival means “left subtree not started”, so you build the thread and go left. The second arrival means “left subtree finished”, so you tear the thread down and finally visit the node. The thread’s presence or absence is the memory. That is what replaces the stack.

12.4 C++

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> out;
    TreeNode* curr = root;

    while (curr != NULL) {
        if (curr->left == NULL) {
            out.push_back(curr->val);
            curr = curr->right;
        }
```

```

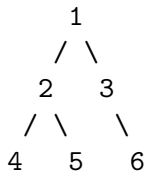
    } else {
        TreeNode* pred = curr->left;
        while (pred->right != NULL && pred->right != curr)
            pred = pred->right;

        if (pred->right == NULL) {           // first visit: thread it and go left
            pred->right = curr;
            curr = curr->left;
        } else {                             // second visit: unthread, visit, go right
            pred->right = NULL;
            out.push_back(curr->val);
            curr = curr->right;
        }
    }
}
return out;
}

```

Twelve lines. No stack, no recursion.

12.5 Dry run



curr=1 has left	pred = rightmost of {2,4,5} = 5, 5.right is NULL thread 5.right = 1, curr = 2
curr=2 has left	pred = rightmost of {4} = 4, 4.right is NULL thread 4.right = 2, curr = 4
curr=4 no left	visit 4 out = [4] curr = 4.right = 2 (thread)
curr=2 has left	pred search: 4.right == 2 == curr, stop unthread 4.right = NULL, visit 2 out = [4,2] curr = 2.right = 5
curr=5 no left	visit 5 out = [4,2,5] curr = 5.right = 1 (thread)
curr=1 has left	pred search: 2 -> 5, 5.right == 1 == curr, stop unthread 5.right = NULL, visit 1 out = [4,2,5,1] curr = 1.right = 3
curr=3 no left	visit 3 out = [4,2,5,1,3]

```

curr = 3.right = 6

curr=6  no left      visit 6      out = [4,2,5,1,3,6]
curr = NULL, loop ends

```

Output 4 2 5 1 3 6, the correct inorder, and the tree is exactly as it was at the start. Every thread that was created was removed.

12.6 Complexity

Space: $O(1)$. Two pointers, curr and pred. Nothing grows with N. The output vector does not count, since it is the answer.

Time: $O(N)$. This surprises people because of the inner while loop that walks down to find the predecessor. The careful argument: every edge of the tree is traversed at most three times, once when searching for a predecessor the first time, once when searching the second time, and once when moving normally. A tree has $N - 1$ edges, so the total work is at most about $3N$ steps. Constant factor larger than plain recursion, same asymptotic class.

12.7 Morris preorder

Preorder needs the node recorded on the **first** arrival instead of the second. So move the push_back into the branch that creates the thread:

```

vector<int> morrisPreorder(TreeNode* root) {
    vector<int> out;
    TreeNode* curr = root;

    while (curr != NULL) {
        if (curr->left == NULL) {
            out.push_back(curr->val);
            curr = curr->right;
        } else {
            TreeNode* pred = curr->left;
            while (pred->right != NULL && pred->right != curr)
                pred = pred->right;

            if (pred->right == NULL) {
                out.push_back(curr->val);    // record on the FIRST visit
                pred->right = curr;
                curr = curr->left;
            } else {
                pred->right = NULL;
                curr = curr->right;
            }
        }
    }
    return out;
}

```

On our tree this gives 1 2 4 5 3 6. Compare it with the inorder version: exactly one line moved. This is the same “first touch versus second touch” idea from the walk-around-the-tree picture in Part 2, now implemented literally.

Morris postorder also exists. It requires reversing the right-spine of a subtree, printing it, and reversing it back. It is fiddly, rarely asked, and not worth memorising at this stage. Know that it exists.

12.8 Risks and honest drawbacks

- **It modifies the tree while running.** If your code throws an exception, returns early, or is interrupted in the middle, the tree is left with dangling threads, which are cycles. Never insert a return in the middle of a Morris loop.
- **It is not safe with concurrent readers.** Another thread reading the tree during traversal sees a corrupted structure.
- **It cannot run on a const tree.** You need write access even though you are only reading.
- **It is harder to debug.** A single wrong condition creates an infinite loop rather than a wrong answer.
- **The constant factor is worse** than plain recursion because of the predecessor searches.

So when is it actually worth it? When the tree is huge and deep enough that $O(H)$ stack space genuinely matters, or when the interviewer asks for $O(1)$ space inorder. In ordinary contest and interview coding, use recursion.

12.9 What you are expected to know

You do not need to reproduce Morris traversal from memory under time pressure. You should be able to:

1. Explain **why** normal traversal needs a stack, in one sentence: there is no way back up.
2. Explain the fix: thread the inorder predecessor’s empty right pointer to the current node.
3. Explain how the code knows whether it is arriving for the first or second time.
4. State the complexities and why time is still $O(N)$.
5. Write the standard implementation with a little thought, having understood it.

12.10 Common mistakes

- **Omitting `pred->right != curr` in the predecessor search.** Infinite loop along an existing thread.
- **Forgetting to remove the thread.** You return the right answer and leave a cycle in the tree, which breaks everything afterwards.
- **Recording the node in the wrong branch.** First branch gives preorder, second gives inorder. Choose deliberately.
- **Moving `curr` to `curr->left` in the unthreading branch.** After unthreading, the left subtree is finished. Go right.
- **Claiming the time is $O(N \log N)$** because of the inner loop. Use the edge counting argument.

- **Using Morris just to look clever** on a problem where recursion is clearer.

12.11 Check yourself

1. What exact problem does the thread solve?
2. Which node do we thread from, and why is its right pointer guaranteed to be free?
3. How does the algorithm distinguish a first arrival from a second arrival at the same node?
4. Why is the time complexity $O(N)$ despite the inner search loop?
5. Which single line moves to convert Morris inorder into Morris preorder?
6. Name two situations where Morris traversal would be a bad choice.

Part 13

How to Think About Tree Problems

Recognising the pattern before writing code

This is the most important part of the course. Everything before it was equipment. This is how you choose which piece of equipment to pick up.

13.1 The ten questions

When you open a tree problem you have never seen, go through this list. It takes about a minute and it decides the entire solution.

- 1. Do I need DFS or BFS?** Does the answer care about levels, distance from the root, or “nearest”? BFS. Does it care about subtree structure? DFS.
- 2. Does the answer at a node depend on its left and right subtrees?** If yes, you are writing `combine(L, R, node)` and the recursion is postorder.
- 3. Should information travel upward or downward?** Upward: return a value (height, count, sum, best-so-far). Downward: add a parameter (depth, running sum, allowed range, path so far). Sometimes both.
- 4. Is this a level based problem?** Any mention of level, row, “each level”, left or right view, zigzag, or level sums means BFS with the size snapshot.
- 5. Am I being asked to avoid recursion?** Then simulate it with an explicit stack. Preorder and inorder templates from Part 4.
- 6. Do I need a horizontal coordinate?** Top, bottom, vertical, column, “looking from above”. Carry `hd` alongside the node, group with a map.
- 7. Is this a path problem, and can the path turn a corner?** Root to leaf, no turn: carry state down. Any node to any node, turns once: bottom-up with the record/return split.
- 8. Is this a subtree problem?** “Contains”, “identical”, “largest subtree such that ...”. Usually a helper comparing two nodes at once, or a bottom-up function returning a bundle of facts.
- 9. Can I decide everything at a single node using only $O(1)$ information from each child?** If yes, one pass is enough and the answer is $O(N)$. If not, decide what larger bundle each child should return.
- 10. Am I repeating work?** Calling a helper that walks a subtree, from inside a recursion that already walks that subtree, is the $O(N^2)$ signature. Merge the two passes.

13.2 The decision guide

Does the problem mention levels, rows, views by level, or "nearest"?

yes -> BFS with the size snapshot template
no -> continue

Does it mention columns, vertical, top view, bottom view?

yes -> BFS carrying horizontal distance + map
no -> continue

Is it a path that may turn at some node (max path sum, diameter)?

yes -> bottom-up: record "through me", return "one side only"
no -> continue

Is it a root-to-leaf path, or does each node need context from above?

yes -> preorder with an extra parameter, backtrack if you keep a shared path
no -> continue

Does my answer need facts about both subtrees?

yes -> postorder: L = solve(left), R = solve(right), combine
no -> plain traversal, do the work at each node

Roughly ninety percent of Easy and Medium tree problems land in one of those five buckets.

13.3 Pattern to problem map

Pattern	Signature in the statement	Typical problems
Simple DFS	count, sum, search, modify every node	Count Nodes, Invert Tree, Merge Trees
Traversal	"return the preorder / inorder"	144, 94, 145
BFS by level	level, row, view, zigzag, average	102, 107, 103, 199, 637, 515
Bottom-up DP	height, depth, balanced, diameter, "longest / best path"	104, 110, 543, 124, 687
Path recursion	root to leaf, path sum, all paths	112, 113, 129, 257
Two-tree recursion	same, mirror, merge, contains	100, 101, 617, 572
Ancestor problems	lowest common ancestor, distance	236, 235, 1123
Horizontal distance	top view, bottom view, vertical	987, 314, GFG views

Construction	build from traversals, serialize	105, 106, 297, 108
BST specific	sorted, kth smallest, validate, search	98, 230, 700, 701, 235

13.4 Applying the checklist to three unseen problems

13.4.1 Example 1: “Return the sum of values of all nodes with a grandparent whose value is even” (LeetCode 1315)

- Levels? No.
- Depends on subtrees? Not really, the answer is a plain sum.
- Information direction? **Downward**. Each node needs to know about its parent and grandparent.
- So: preorder with two extra parameters.

```
int dfs(TreeNode* root, int parent, int grandparent) {
    if (root == NULL) return 0;
    int add = (grandparent % 2 == 0) ? root->val : 0;
    return add + dfs(root->left, root->val, parent)
        + dfs(root->right, root->val, parent);
}
// call dfs(root, 1, 1) using odd sentinels so the top two levels contribute nothing
```

The whole problem was solved by question 3.

13.4.2 Example 2: “Find the largest subtree that is a BST”

- Levels? No.
- Depends on subtrees? Yes, strongly.
- Can each child give me O(1) information? Let us see. To know whether I am a BST I need: is my left subtree a BST, is my right subtree a BST, is the **maximum** of my left subtree less than me, is the **minimum** of my right subtree greater than me. And to report a size I need their sizes.
- So each node returns four things: {isBST, minVal, maxVal, size}.

That is question 9 doing its job. The moment you list what you need, the return type designs itself, and the code is then mechanical.

13.4.3 Example 3: “Print all nodes at distance K from a target node”

- Levels? Yes, but distance from an arbitrary node, not from the root.
- The obstacle: distance goes both down and **up**, and a tree has no parent pointers.
- Fix: one preorder pass to record each node’s parent in a map, then BFS outward from the target treating the tree as an undirected graph, with a visited set.

Question 3 exposed the obstacle (information must move upward through edges that only point down), and the fix follows immediately. This is LeetCode 863, a Medium that many people find hard purely because they never asked which direction the information travels.

13.5 When you are stuck

In order, these are the moves that work:

1. **Draw a tree with 5 to 7 nodes and solve it by hand.** Not in your head. On paper. Then ask what you actually did.
2. **Write down what you wish each subtree would tell you.** Then that is the return type.
3. **Answer the base case honestly.** What is this quantity for an empty tree?
4. **Solve the two node case, then the three node case.** Bugs show up immediately.
5. **Say the function's meaning out loud in one sentence.** If you cannot, the code is guesswork. “f(x) returns the height of x's subtree, or -1 if it is unbalanced” is a sentence. “f does the balance thing” is not.
6. **Check for repeated work.** If a helper walks the subtree inside a recursion that walks the subtree, merge them.

13.6 Habits that separate strong students

- They write the base case first, always.
- They name the return value before writing the body.
- They keep “the answer through this node” and “the value returned to my parent” as two separate variables, even when they happen to be equal.
- They test on a skewed tree, a single node, and an empty tree, not just the pretty balanced example in the problem statement.
- They reset globals at the start of the public function.
- They state the complexity out loud before submitting, and check that it matches the constraints.

13.7 Check yourself

1. A problem asks for the average value of the nodes on each level. Which bucket is it, and what template do you reach for?
2. A problem asks for the length of the longest path where all node values are equal. Which bucket, and what does each node return upward?
3. A problem asks whether a tree contains a root to leaf path whose values spell a given word. Which direction does information flow?
4. A problem asks for the node values visible when looking at the tree from directly above. What coordinate do you need, and what traversal?
5. You have written a solution that calls `height()` at every node. What should you check before submitting?
6. Why is “state the meaning of the return value in one sentence” such a useful discipline?

Part 14

Practice Problem Sets

Graded, with hints and no solutions

14.1 How to use this set

Rules that make the difference between practice and time-wasting:

1. **Answer the “think about” line before you write any code.** Out loud or on paper. That line is the actual exercise. The typing is not.
2. **Give yourself 20 minutes** on an Easy, 35 on a Medium. If you are stuck after that, read a hint, not a solution.
3. **After you solve it, say the pattern name out loud.** “That was bottom-up with a record and return split.” Naming is what makes the next problem faster.
4. **Redo, do not collect.** Solving 40 problems once teaches less than solving 20 problems twice, a week apart.
5. **Test on the ugly cases:** empty tree, single node, skewed tree, all negative values.

No solutions are printed here on purpose.

14.2 Level 1: Absolute Beginner

Get the template into your fingers. Every one of these is under ten lines.

- **Binary Tree Preorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-preorder-traversal/> Pattern: recursive DFS. Think: where does push_back go relative to the two calls, and why does the output vector need &?
- **Binary Tree Inorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-inorder-traversal/> Pattern: recursive DFS. Think: predict the output on our sample tree before coding.
- **Binary Tree Postorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-postorder-traversal/> Pattern: recursive DFS. Think: why is the root always last?
- **Maximum Depth of Binary Tree** (Easy). <https://leetcode.com/problems/maximum-depth-of-binary-tree/> Pattern: bottom-up. Think: what is the depth of an empty tree, and what do you add for yourself?
- **Sum of Left Leaves** (Easy). <https://leetcode.com/problems/sum-of-left-leaves/> Pattern: DFS with a flag. Think: a node cannot tell whether it is a left child. Who knows that, and how do you tell it?
- **Univalued Binary Tree** (Easy). <https://leetcode.com/problems/univalued-binary-tree/> Pattern: DFS returning a boolean. Think: what should NULL return, and why is it not false?

- **Search in a Binary Search Tree** (Easy). <https://leetcode.com/problems/search-in-a-binary-search-tree/> Pattern: BST descent. Think: why do you never need to look at both children?
- **Binary Tree Level Order Traversal** (Medium by label, Easy in practice). <https://leetcode.com/problems/binary-tree-level-order-traversal/> Pattern: BFS template. Think: why must the level size be snapshotted before the inner loop?

14.3 Level 2: Easy

Standard interview warm-ups. You should reach a point where these take under ten minutes each.

- **Invert Binary Tree** (Easy). <https://leetcode.com/problems/invert-binary-tree/> Pattern: simple DFS with modification. Think: if you swap first and then recurse, what must you be careful about?
- **Same Tree** (Easy). <https://leetcode.com/problems/same-tree/> Pattern: two-tree recursion. Think: what are the three base cases, and in which order must they be tested?
- **Symmetric Tree** (Easy). <https://leetcode.com/problems/symmetric-tree/> Pattern: two-tree recursion, mirrored. Think: why can you not just call Same Tree on the two halves?
- **Minimum Depth of Binary Tree** (Easy). <https://leetcode.com/problems/minimum-depth-of-binary-tree/> Pattern: DFS with a trap, or BFS with early exit. Think: what goes wrong on a tree that is one long right chain?
- **Path Sum** (Easy). <https://leetcode.com/problems/path-sum/> Pattern: top-down path. Think: where exactly is the target checked, at NULL or at a leaf? Why does it matter?
- **Balanced Binary Tree** (Easy). <https://leetcode.com/problems/balanced-binary-tree/> Pattern: bottom-up with a sentinel. Think: what is the cost of the naive version, and what single value can carry both height and failure?
- **Diameter of Binary Tree** (Easy). <https://leetcode.com/problems/diameter-of-binary-tree/> Pattern: bottom-up, record versus return. Think: which expression is recorded and which is returned?
- **Merge Two Binary Trees** (Easy). <https://leetcode.com/problems/merge-two-binary-trees/> Pattern: two-tree recursion. Think: what should happen when one of the two nodes is NULL?
- **Binary Tree Paths** (Easy). <https://leetcode.com/problems/binary-tree-paths/> Pattern: backtracking. Think: what must be undone when leaving a node?
- **Average of Levels in Binary Tree** (Easy). <https://leetcode.com/problems/average-of-levels-in-binary-tree/> Pattern: BFS template. Think: which type do you accumulate in, and why not int?

14.4 Level 3: Easy plus

Each of these needs two ideas at once, or an idea used slightly out of its comfort zone.

- **Binary Tree Level Order Traversal II** (Medium). <https://leetcode.com/problems/binary-tree-level-order-traversal-ii/> Pattern: BFS plus a final reversal. Think: is there any reason to be clever here?
- **Binary Tree Right Side View** (Medium). <https://leetcode.com/problems/binary-tree-right-side-view/> Pattern: BFS last of level, or DFS visiting right first. Think: why is “keep going right from the root” wrong?

- **Find Bottom Left Tree Value** (Medium). <https://leetcode.com/problems/find-bottom-left-tree-value/> Pattern: BFS. Think: is there a queue order that makes this a one-liner?
- **Sum Root to Leaf Numbers** (Medium). <https://leetcode.com/problems/sum-root-to-leaf-numbers/> Pattern: top-down accumulation. Think: what number do you pass down, and what does NULL contribute?
- **Path Sum II** (Medium). <https://leetcode.com/problems/path-sum-ii/> Pattern: backtracking plus target. Think: two things change as you descend. What are they?
- **Count Good Nodes in Binary Tree** (Medium). <https://leetcode.com/problems/count-good-nodes-in-binary-tree/> Pattern: top-down with carried maximum. Think: what does each node need to be told by its parent?
- **Subtree of Another Tree** (Easy). <https://leetcode.com/problems/subtree-of-another-tree/> Pattern: two-tree recursion inside a traversal. Think: why is this one of the rare cases where calling a helper at every node is acceptable?
- **Cousins in Binary Tree** (Easy). <https://leetcode.com/problems/cousins-in-binary-tree/> Pattern: BFS or DFS with depth and parent. Think: two facts must match and one must differ. Which?
- **Maximum Level Sum of a Binary Tree** (Medium). <https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree/> Pattern: BFS with an accumulator. Think: what tie-breaking rule does the problem impose?

14.5 Level 4: Medium

These are the recurring interview patterns. Do not skip the “think about” lines here.

- **Binary Tree Zigzag Level Order Traversal** (Medium). <https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/> Pattern: BFS plus alternate reversal. Think: what happens if you try to alternate by pushing children in a different order instead?
- **Lowest Common Ancestor of a Binary Tree** (Medium). <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/> Pattern: bottom-up search. Think: state in one sentence what the function returns for an arbitrary node. Do not write code before you can.
- **Lowest Common Ancestor of a BST** (Medium). <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/> Pattern: BST descent. Think: what property identifies the split point?
- **Construct Binary Tree from Preorder and Inorder Traversal** (Medium). <https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/> Pattern: divide and conquer with index bookkeeping. Think: which array gives the root, which gives the split, and why must you build left before right?
- **Construct Binary Tree from Inorder and Postorder Traversal** (Medium). <https://leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal/> Pattern: same, mirrored. Think: which end of postorder do you consume, and which subtree comes first now?
- **Validate Binary Search Tree** (Medium). <https://leetcode.com/problems/validate-binary-search-tree/> Pattern: range passed downward, or inorder check. Think: build a tree where the local check succeeds and the tree is not a BST.
- **Kth Smallest Element in a BST** (Medium). <https://leetcode.com/problems/kth-smallest-element-in-a-bst/> Pattern: inorder with a counter. Think: can you stop early? What does that do to the complexity?

- **Binary Tree Maximum Path Sum** (Hard, but a must-know). <https://leetcode.com/problems/binary-tree-maximum-path-sum/> Pattern: bottom-up, record versus return. Think: why $\max(0, \text{child})$, and why must the answer start at `INT_MIN`?
- **Path Sum III** (Medium). <https://leetcode.com/problems/path-sum-iii/> Pattern: prefix sums on a path. Think: what does the map hold, and when is an entry removed?
- **All Nodes Distance K in Binary Tree** (Medium). <https://leetcode.com/problems/all-nodes-distance-k-in-binary-tree/> Pattern: parent map plus BFS. Think: why can plain DFS from the target not work directly?
- **Vertical Order Traversal of a Binary Tree** (Hard label, Medium idea). <https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/> Pattern: horizontal distance plus sorting. Think: what are the three tie-breaking keys, in order?
- **Serialize and Deserialize Binary Tree** (Hard label, Medium idea). <https://leetcode.com/problems/serialize-and-deserialize-binary-tree/> Pattern: preorder with null markers. Think: why does a plain traversal without markers lose the tree?
- **Flatten Binary Tree to Linked List** (Medium). <https://leetcode.com/problems/flatten-binary-tree-to-linked-list/> Pattern: preorder rearrangement. Think: which traversal order does the flattened list follow, and where do you attach the old right subtree?
- **Maximum Width of Binary Tree** (Medium). <https://leetcode.com/problems/maximum-width-of-binary-tree/> Pattern: BFS carrying an index. Think: indices grow like 2^i and 2^{i+1} . What goes wrong for a deep tree, and how do you fix it?
- **Longest Univalue Path** (Medium). <https://leetcode.com/problems/longest-univalue-path/> Pattern: diameter skeleton with a condition. Think: when does a child contribute nothing?

14.6 The three view problems

These are standard in Indian campus interviews and on GeeksforGeeks rather than LeetCode. They are worth doing.

- **Top View of Binary Tree.** <https://www.geeksforgeeks.org/problems/top-view-of-binary-tree/1> Pattern: BFS with horizontal distance, first node per column.
- **Bottom View of Binary Tree.** <https://www.geeksforgeeks.org/problems/bottom-view-of-binary-tree/1> Pattern: same, last node per column.
- **Boundary Traversal of Binary Tree.** <https://www.geeksforgeeks.org/problems/boundary-traversal-of-binary-tree/1> Pattern: three separate walks. Think: where can duplicates appear?

14.7 Must Solve

If you only have time for one list, this is it. Twenty problems covering every pattern in the course. Solve all of them, then redo the starred ones a week later.

#	Problem	Pattern tested
1	Binary Tree Preorder Traversal (144)	preorder, recursion
2	Binary Tree Inorder Traversal (94) *	inorder, iterative too
3	Binary Tree Postorder Traversal (145)	postorder, iterative too

4	Binary Tree Level Order Traversal (102) *	BFS template
5	Maximum Depth of Binary Tree (104)	bottom-up basics
6	Minimum Depth of Binary Tree (111)	the leaf trap
7	Invert Binary Tree (226)	simple DFS with change
8	Same Tree (100)	two-tree recursion
9	Symmetric Tree (101) *	mirror recursion
10	Balanced Binary Tree (110) *	sentinel, $O(N)$ rewrite
11	Diameter of Binary Tree (543) *	record versus return
12	Path Sum (112)	top-down path
13	Path Sum II (113)	backtracking
14	Binary Tree Paths (257)	backtracking, undo
15	Binary Tree Right Side View (199)	level based view
16	Binary Tree Zigzag Level Order (103)	BFS variation
17	Lowest Common Ancestor (236) *	ancestor recursion
18	Construct from Preorder and Inorder (105) *	construction
19	Binary Tree Maximum Path Sum (124) *	bottom-up, hard version
20	Vertical Order Traversal (987)	horizontal distance
21	Top View of Binary Tree (GFG)	horizontal distance
22	Bottom View of Binary Tree (GFG)	horizontal distance
23	Boundary Traversal (GFG)	three-part walk
24	Validate Binary Search Tree (98)	range passed downward
25	Serialize and Deserialize (297)	preorder with markers

Coverage check: DFS, BFS, all three traversal orders, iterative traversal (problems 2 and 3, do both ways), height, diameter, balance, paths, symmetry, inversion, LCA, views, vertical order, construction, BST, and bottom-up thinking. If you can solve all twenty-five without looking anything up, you know binary trees.

14.8 A four week plan

Week	Sections to study	Problems
1	Parts 1, 2, 3	Level 1, plus Must Solve 1 to 6
2	Parts 4, 5, 6	Level 2, plus Must Solve 7 to 11
3	Parts 7, 8, 9, 10	Level 3, plus Must Solve 12 to 19
4	Parts 11, 12, 13	Level 4, plus Must Solve 20 to 25

Then redo the starred problems from memory. If any of them takes more than 15 minutes, reread the relevant part.

Part 15

Revision Sheet

Everything on a few pages, for the night before

15.1 Terminology in one box

root	topmost node, no parent
leaf	BOTH children are NULL
internal	not a leaf
ancestor	any node above you on the path to the root
subtree	a node plus ALL of its descendants
level	distance from the root, root is level 0
depth	edges from the root down to the node (measured downward from root)
height	edges from the node down to its deepest leaf (measured upward from leaves)

N nodes => N - 1 edges

perfect tree of height h => $2^{(h+1)} - 1$ nodes, level d holds 2^d nodes

balanced => height about $\log_2(N)$

skewed => height N - 1

BST: for EVERY node, whole left subtree < node < whole right subtree
=> inorder of a BST is sorted

Height counting convention: with $h(\text{NULL}) = 0$ you are counting nodes, with $h(\text{NULL}) = -1$ you are counting edges. LeetCode counts nodes.

15.2 Node definition

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
```

15.3 The master template

```
ReturnType solve(TreeNode* root) {
    if (root == NULL) return answerForEmptyTree;
```

```

    ReturnType L = solve(root->left);
    ReturnType R = solve(root->right);
    return combine(L, R, root->val);
}

```

Base case identities:

```

sum -> 0          count -> 0          max -> INT_MIN      min -> INT_MAX
all -> true       any -> false       height(nodes) -> 0  height(edges) -> -1

```

15.4 Recursive traversals

```

void pre(TreeNode* r, vector<int>& v) {
    if (!r) return;
    v.push_back(r->val); pre(r->left, v); pre(r->right, v);
}
void in(TreeNode* r, vector<int>& v) {
    if (!r) return;
    in(r->left, v); v.push_back(r->val); in(r->right, v);
}
void post(TreeNode* r, vector<int>& v) {
    if (!r) return;
    post(r->left, v); post(r->right, v); v.push_back(r->val);
}

```

On the standard tree:

```

      1
     /\
    2 3
   /\ \
  4 5 6
pre  : 1 2 4 5 3 6
in   : 4 2 5 1 3 6
post : 4 5 2 6 3 1
bfs  : 1 2 3 4 5 6

```

15.5 Iterative traversals

```

// PREORDER: pop, record, push right then left
stack<TreeNode*> st; st.push(root);
while (!st.empty()) {
    TreeNode* n = st.top(); st.pop();
    out.push_back(n->val);
    if (n->right) st.push(n->right);
    if (n->left)  st.push(n->left);
}

// INORDER: dive left, pop, record, go right
stack<TreeNode*> st; TreeNode* cur = root;
while (cur || !st.empty()) {
    while (cur) { st.push(cur); cur = cur->left; }
    cur = st.top(); st.pop();
    out.push_back(cur->val);
}

```

```

    cur = cur->right;
}

// POSTORDER: preorder with left/right swapped, then reverse
stack<TreeNode*> st; st.push(root);
while (!st.empty()) {
    TreeNode* n = st.top(); st.pop();
    out.push_back(n->val);
    if (n->left) st.push(n->left);
    if (n->right) st.push(n->right);
}
reverse(out.begin(), out.end());

```

15.6 BFS template

```

vector<vector<int>>> levelOrder(TreeNode* root) {
    vector<vector<int>>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);

    while (!q.empty()) {
        int sz = q.size();                // snapshot, never q.size() in the condition
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop();
            level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        res.push_back(level);
    }
    return res;
}

```

Variations, all inside the inner loop:

```

right view    : if (i == sz - 1) res.push_back(n->val);
left view     : if (i == 0)      res.push_back(n->val);
level sum     : sum += n->val;
level max     : mx = max(mx, n->val);
zigzag        : build level normally, reverse it on alternate rows
bottom-up     : build normally, reverse(res) at the end

```

15.7 Height template

```

int height(TreeNode* r) {                // in nodes
    if (!r) return 0;
    return 1 + max(height(r->left), height(r->right));
}

```

15.8 Bottom-up template

```
int dfs(TreeNode* r) {
    if (!r) return 0;
    int L = dfs(r->left);
    int R = dfs(r->right);
    best = max(best, /* answer THROUGH r, uses L and R */);
    return /* value for the PARENT, uses only one of L, R */;
}
```

15.9 Diameter template

```
int best = 0;
int height(TreeNode* r) {
    if (!r) return 0;
    int L = height(r->left), R = height(r->right);
    best = max(best, L + R); // edges of the path turning here
    return 1 + max(L, R);
}
// answer = best
```

15.10 Balanced template

```
int check(TreeNode* r) { // height, or -1 if unbalanced
    if (!r) return 0;
    int L = check(r->left); if (L == -1) return -1;
    int R = check(r->right); if (R == -1) return -1;
    if (abs(L - R) > 1) return -1;
    return 1 + max(L, R);
}
// balanced <=> check(root) != -1
```

15.11 Path templates

```
// root to leaf, target
bool hasPathSum(TreeNode* r, int t) {
    if (!r) return false;
    t -= r->val;
    if (!r->left && !r->right) return t == 0;
    return hasPathSum(r->left, t) || hasPathSum(r->right, t);
}

// all root to leaf paths, with backtracking
void dfs(TreeNode* r, vector<int>& path, vector<vector<int>>& res) {
    if (!r) return;
    path.push_back(r->val);
    if (!r->left && !r->right) res.push_back(path);
}
```

```

    else { dfs(r->left, path, res); dfs(r->right, path, res); }
    path.pop_back(); // undo
}

// maximum path sum, any node to any node
int gain(TreeNode* r) {
    if (!r) return 0;
    int L = max(0, gain(r->left));
    int R = max(0, gain(r->right));
    best = max(best, r->val + L + R); // record
    return r->val + max(L, R); // return
}
// best starts at INT_MIN

```

15.12 View templates

```

// TOP / BOTTOM view: BFS carrying horizontal distance
map<int,int> col;
queue<pair<TreeNode*,int>> q; q.push({root, 0});
while (!q.empty()) {
    auto c = q.front(); q.pop();
    TreeNode* n = c.first; int hd = c.second;

    if (col.find(hd) == col.end()) col[hd] = n->val; // TOP: first wins
    // col[hd] = n->val; // BOTTOM: last wins

    if (n->left) q.push({n->left, hd - 1});
    if (n->right) q.push({n->right, hd + 1});
}
// read col in key order

// VERTICAL order: map<int, vector<int>> col; col[hd].push_back(n->val);

// RIGHT view by DFS: visit right first, record when level == res.size()

```

```

// BOUNDARY traversal
bool isLeaf(TreeNode* n) { return !n->left && !n->right; }

// 1. root, if it is not a leaf
// 2. left boundary top down, skipping leaves
//     cur = root->left; go left if possible, else right
// 3. all leaves, left to right (simple DFS)
// 4. right boundary bottom up, skipping leaves
//     cur = root->right; go right if possible, else left; collect then reverse

```

Horizontal distance rule: $hd(\text{left}) = hd - 1$, $hd(\text{right}) = hd + 1$, $hd(\text{root}) = 0$.

15.13 Morris template

```

TreeNode* cur = root;
while (cur) {
    if (!cur->left) {
        out.push_back(cur->val);           // preorder records here too
        cur = cur->right;
    } else {
        TreeNode* pred = cur->left;
        while (pred->right && pred->right != cur) pred = pred->right;

        if (!pred->right) {
            // out.push_back(cur->val);     // uncomment for PREORDER
            pred->right = cur;             // create thread
            cur = cur->left;
        } else {
            pred->right = NULL;            // remove thread
            out.push_back(cur->val);       // INORDER records here
            cur = cur->right;
        }
    }
}

```

$O(N)$ time, $O(1)$ extra space, temporarily modifies the tree.

15.14 Complexity cheat sheet

Algorithm	Time	Space	Note
Recursive traversal	$O(N)$	$O(H)$	call stack
Iterative traversal	$O(N)$	$O(H)$	explicit stack
Morris traversal	$O(N)$	$O(1)$	modifies tree temporarily
BFS level order	$O(N)$	$O(W)$	W is max width, up to $N/2$
Height, count, sum	$O(N)$	$O(H)$	one pass
Diameter, naive	$O(N^2)$	$O(H)$	height recomputed
Diameter, bottom-up	$O(N)$	$O(H)$	
Balanced, naive	$O(N^2)$	$O(H)$	
Balanced, sentinel	$O(N)$	$O(H)$	
All root to leaf paths	$O(N \cdot H)$	$O(H)$	copying each path
Max path sum	$O(N)$	$O(H)$	
Views with a map	$O(N \log N)$	$O(N)$	$O(N)$ with a vector and offset
Boundary traversal	$O(N)$	$O(H)$	
Build from preorder + inorder	$O(N)$	$O(N)$	needs the index map
Serialize / deserialize	$O(N)$	$O(N)$	
Subtree of another tree	$O(N \cdot M)$	$O(H)$	
BST search, insert, LCA	$O(H)$	$O(1)$ iterative	$O(\log N)$ if balanced

H is the height: $O(\log N)$ balanced, $O(N)$ skewed.

15.15 Which approach should I think of first

1. Read the statement. Circle the words: level, view, path, subtree, ancestor, column, nearest, sorted, BST.
2. levels / rows / views by level / nearest
 top / bottom / vertical / column
 path that may turn
 root to leaf / needs context from above
 needs facts from both subtrees
 sorted / kth / validate / search
 "without recursion"
 "O(1) extra space" and inorder
 - > BFS, size snapshot
 - > BFS + horizontal distance + map
 - > bottom-up, record vs return
 - > preorder + extra parameter
 - > postorder, combine(L, R, node)
 - > BST, use inorder or the ordering
 - > explicit stack
 - > Morris
3. Write the base case first. What is the answer for an empty tree?
4. Say in one sentence what the function returns.
5. Ask: am I calling a subtree-walking helper inside a recursion that already walks the subtree? If yes, merge the two passes.
6. State the complexity. Compare with the constraints. Then code.

15.16 The mistakes that cost the most marks

missing NULL base case
 leaf test written with || instead of &&
 "missing child" treated as a leaf (minimum depth)
 target checked at NULL instead of at a leaf (path sum)
 missing & on the output vector
 q.size() used in the loop condition instead of a snapshot
 returning L + R upward in diameter or max path sum
 best initialised to 0 instead of INT_MIN in max path sum
 recomputing height at every node -> $O(N^2)$
 forgetting pop_back in backtracking
 global not reset between test cases
 unordered_map used where the output must be ordered by column
 right boundary not reversed
 root printed twice in boundary traversal
 Morris thread not removed
 assuming a plain binary tree is a BST

15.17 The four questions to ask on any new tree problem

1. What information does each subtree need to return?

2. Is this DFS, BFS, or bottom-up?
3. What should my base case be?
4. What do I combine from the left and the right?

If you can answer these four for a problem you have never seen, you have learned binary trees.